ArkLib: Compositional Verification of SNARKs in Lean

Quang Dao
CMU

In collaboration with: **Ethereum Foundation** (Alexander Hicks, Derek Sorensen), Devon Tuma, **Nethermind** (Julian Sutherland, Katy Hristova, Ilia Vlasov, Frantisek Silvasi), Chung Nguyen, **zkSecurity** (Gregor Mitscha-Baude), Bolton Bailey, and many others

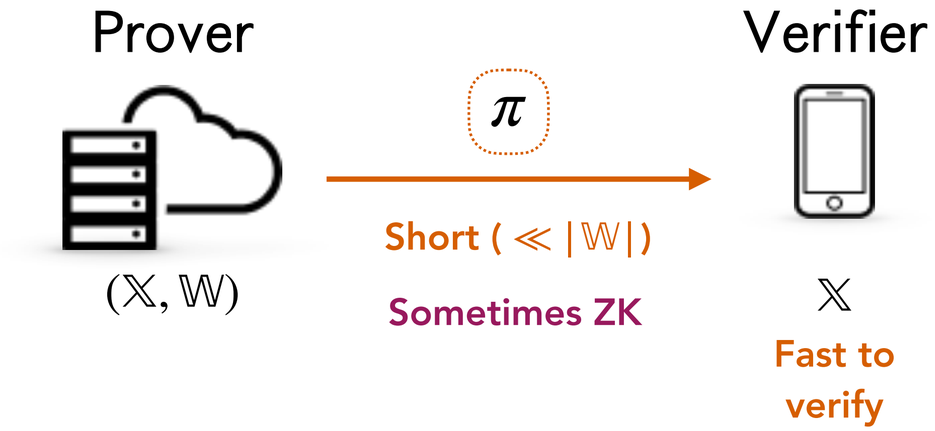
Real World Cryptography, March 9 2026

SNARKs Secure Billions of Dollars, Today

(Succinct Non-interactive ARguments of Knowledge)

SNARKs Secure Billions of Dollars, Today

(Succinct Non-interactive ARguments of Knowledge)



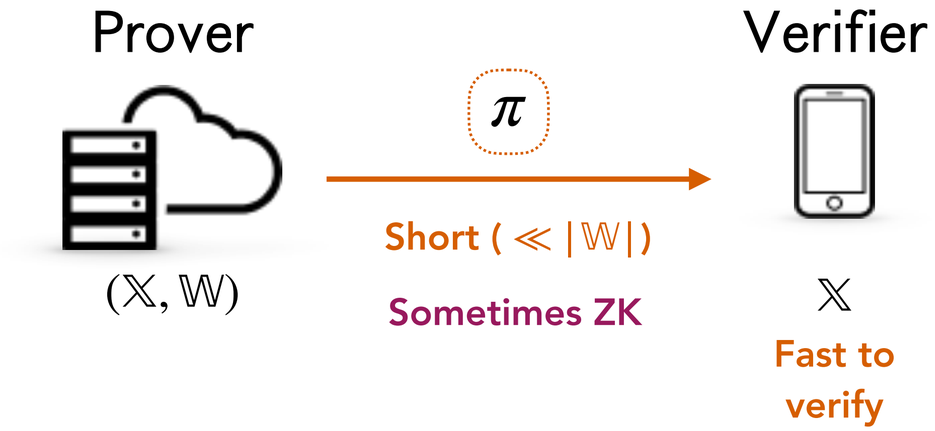
"My digital ID shows that I am above 18"

"I claim that this is the correct next Ethereum block"

"This AI model returns this result when given these inputs"

SNARKs Secure Billions of Dollars, Today

(Succinct Non-interactive ARguments of Knowledge)



"My digital ID shows that I am above 18"

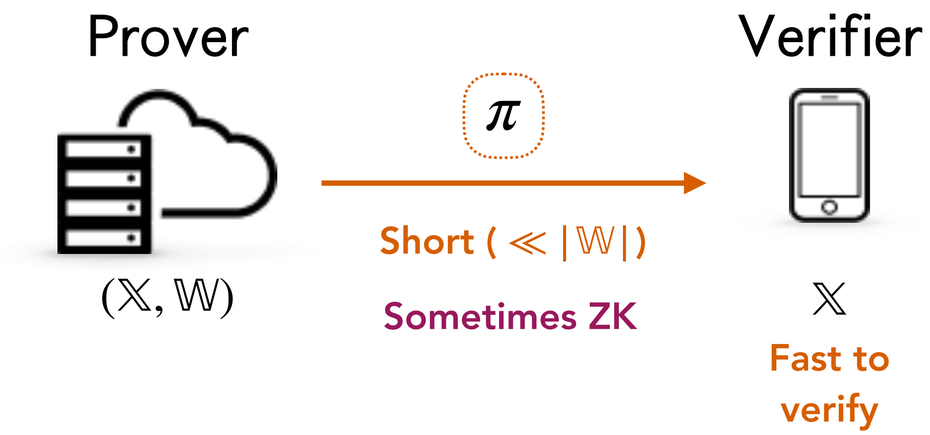
"I claim that this is the correct next Ethereum block"

"This AI model returns this result when given these inputs"

Critical infrastructure like TLS, MLS, etc.

SNARKs Secure Billions of Dollars, Today

(Succinct Non-interactive ARguments of Knowledge)



"My digital ID shows that I am above 18"

"I claim that this is the correct next Ethereum block"

"This AI model returns this result when given these inputs"

Critical infrastructure like TLS, MLS, etc.

Deployed SNARKs need formal verification!

- Constraints
- Protocols
- Implementations

The Concerning State of SNARKs

The Concerning State of SNARKs

Bugs are common
& often catastrophic

Coordinated disclosure of vulnerabilities affecting Girault, Bulletproofs, and PlonK

Zcash Team Reveals It Fixed a Catastrophic Coin Counterfeiting Bug

The First ZK Exploits Happened, and They Weren't What We Expected

MAR 3, 2026

Unfaithful Claims: Breaking 6 zkVMs

The Concerning State of SNARKs

Bugs are common
& often catastrophic

Coordinated disclosure of vulnerabilities affecting Girault, Bulletproofs, and PlonK

Zcash Team Reveals It Fixed a Catastrophic Coin Counterfeiting Bug

The First ZK Exploits Happened, and They Weren't What We Expected

MAR 3, 2026

Unfaithful Claims: Breaking 6 zkVMs

Implementation differs
from spec (if one exists)

stwo risc0 sp1 openvm zisk ...

Scalable, transparent, and post-quantum secure computational integrity

Eli Ben-Sasson* Iddo Bentov† Yinon Horesh* Michael Riabzev*

jolt

Jolt: SNARKs for Virtual Machines via Lookups

Arasu Arun* Srinath Setty† Justin Thaler‡

binius64

Succinct Arguments over Towers of Binary Fields

Benjamin E. Diamond Jim Posen

The Concerning State of SNARKs

Bugs are common
& often catastrophic

Coordinated disclosure of vulnerabilities affecting Girault, Bulletproofs, and PlonK

Zcash Team Reveals It Fixed a Catastrophic Coin Counterfeiting Bug

The First ZK Exploits Happened, and They Weren't What We Expected

MAR 3, 2026

Unfaithful Claims: Breaking 6 zkVMs

Implementation differs
from spec (if one exists)

stwo risc0 sp1 openvm zisk ...

Scalable, transparent, and post-quantum secure computational integrity

Eli Ben-Sasson* Iddo Bentov† Yinon Horesh* Michael Riabzev*

jolt

Jolt: SNARKs for Virtual Machines via Lookups

Arasu Arun* Srinath Setty† Justin Thaler‡

binius64

Succinct Arguments over Towers of Binary Fields

Benjamin E. Diamond Jim Posen

Literature is
scattered & incomplete

Building Cryptographic Proofs from Hash Functions

A Fiat–Shamir Transformation From Duplex Sponges

Proofs, Arguments, and Zero-Knowledge

Desiderata for Formally Verified SNARKs

Desiderata for Formally Verified SNARKs

Spec $\Longleftrightarrow$ Implementation

Desiderata for Formally Verified SNARKs

Spec $\Longleftrightarrow$ Implementation

Consolidated, modular definitions & theorems

Desiderata for Formally Verified SNARKs

Spec $\Longleftrightarrow$ Implementation

Consolidated, modular definitions & theorems

Build fresh from high-assurance foundation

ArkLib: Verified SNARKs in Lean

ArkLib: Verified SNARKs in Lean

Framework for formal specifications & security proofs of SNARKs:

- Building blocks (Interactive Oracle Reductions) + composition patterns
- Executable spec with links to production implementations

ArkLib: Verified SNARKs in Lean

Framework for formal specifications & security proofs of SNARKs:

- Building blocks (Interactive Oracle Reductions) + composition patterns
- Executable spec with links to production implementations

Funded by Ethereum Foundation

- Active open-source contributions

zkEVM Formal Verification Project

Accelerating the application of formal verification methods to zkEVMs to ensure correctness and security.

ArkLib: Verified SNARKs in Lean

Framework for formal specifications & security proofs of SNARKs:

- Building blocks (Interactive Oracle Reductions) + composition patterns
- Executable spec with links to production implementations

Funded by Ethereum Foundation

- Active open-source contributions

zkEVM Formal Verification Project

Accelerating the application of formal verification methods to zkEVMs to ensure correctness and security.

Lean is an open-source programming language and proof assistant that enables correct, maintainable, and formally verified code

The Mathlib Initiative

Building the Digital Foundation of Mathematics

CLAUDE CODE

Codex

Harmonic

Axiom

Math, Inc.

Logical Intelligence

Roadmap

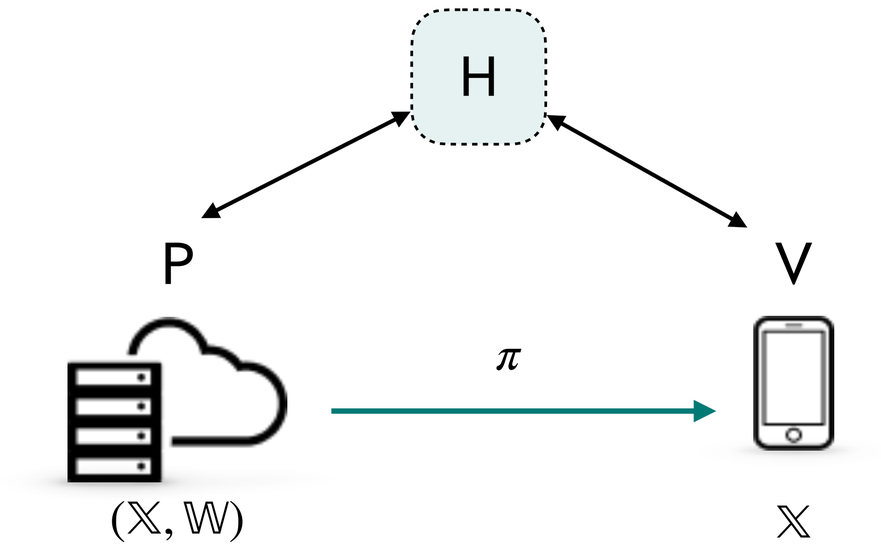
1. SNARKs from Interactive Oracle Reductions

2. Walkthrough of ArkLib

3. Future Plans & AI for Verified Cryptography

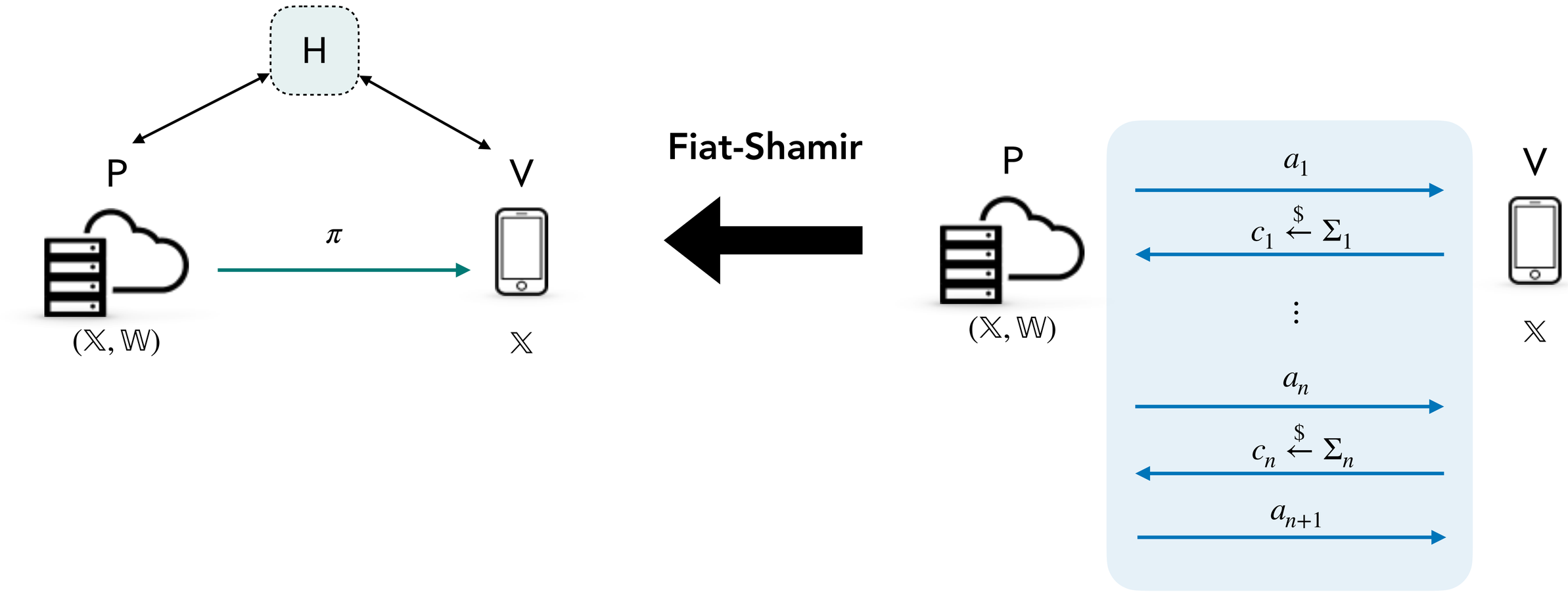
Anatomy of a SNARK

Anatomy of a SNARK



Non-Interactive Argument

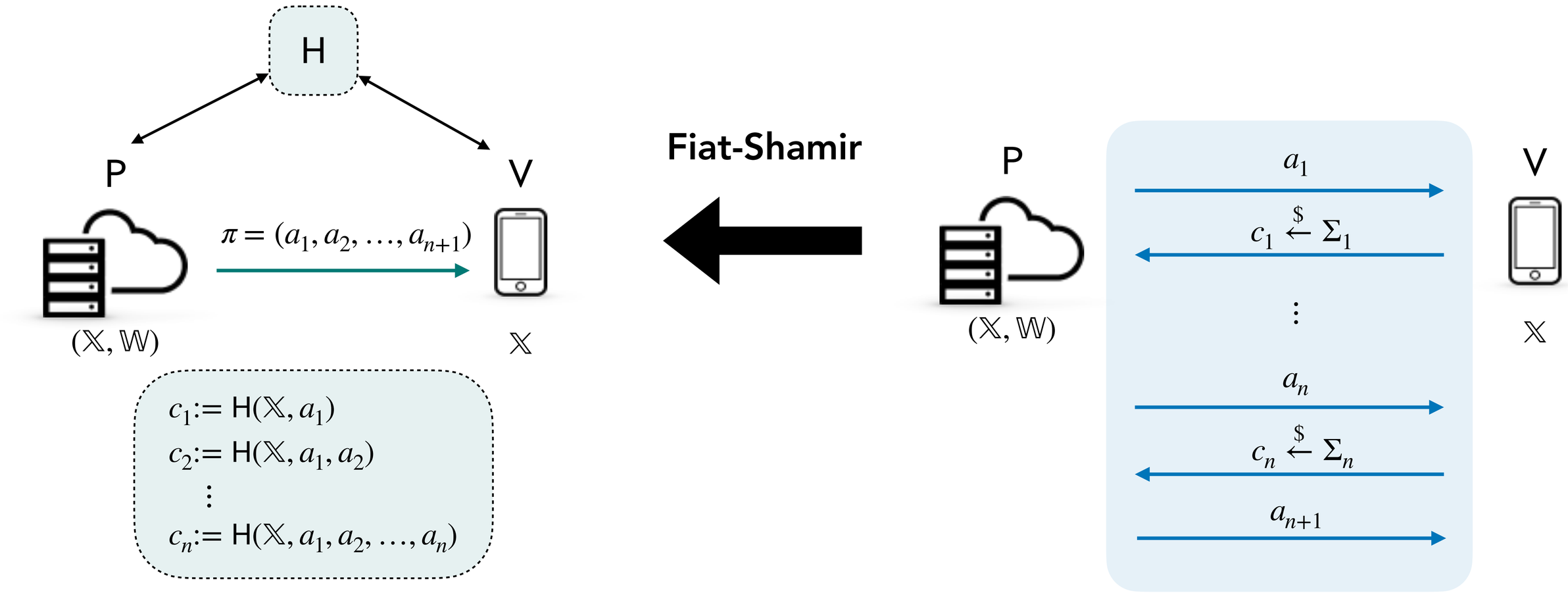
Anatomy of a SNARK



Non-Interactive Argument

(Public-Coin) Interactive Argument

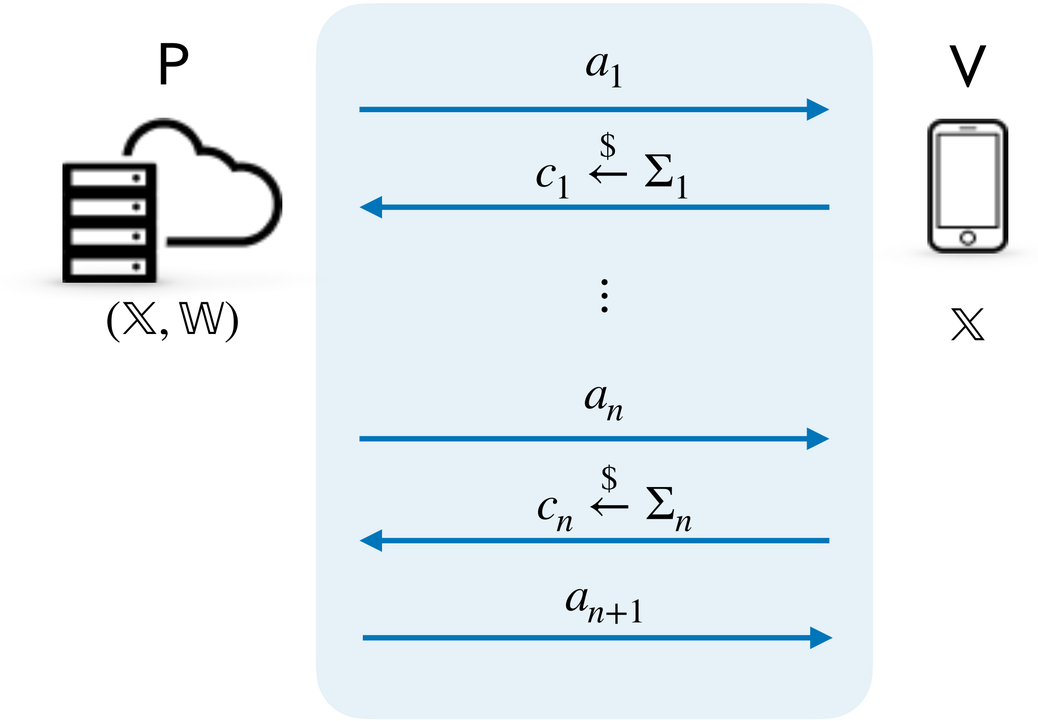
Anatomy of a SNARK



Non-Interactive Argument

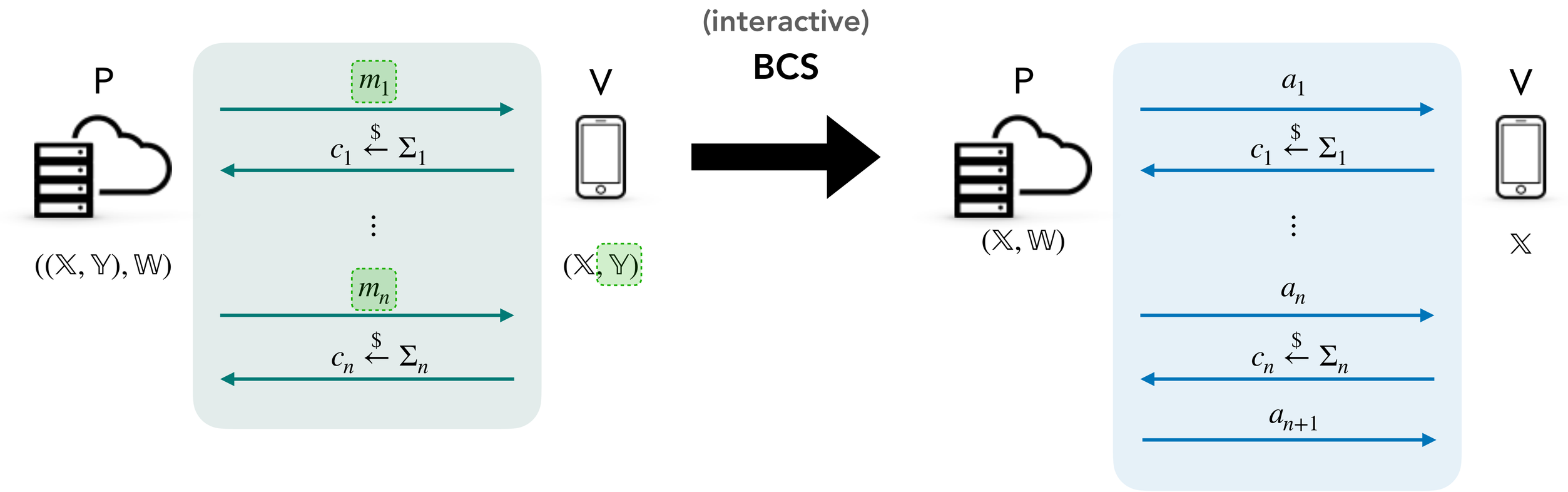
(Public-Coin) Interactive Argument

Anatomy of a SNARK



Interactive Argument

Anatomy of a SNARK



Interactive Oracle Proof

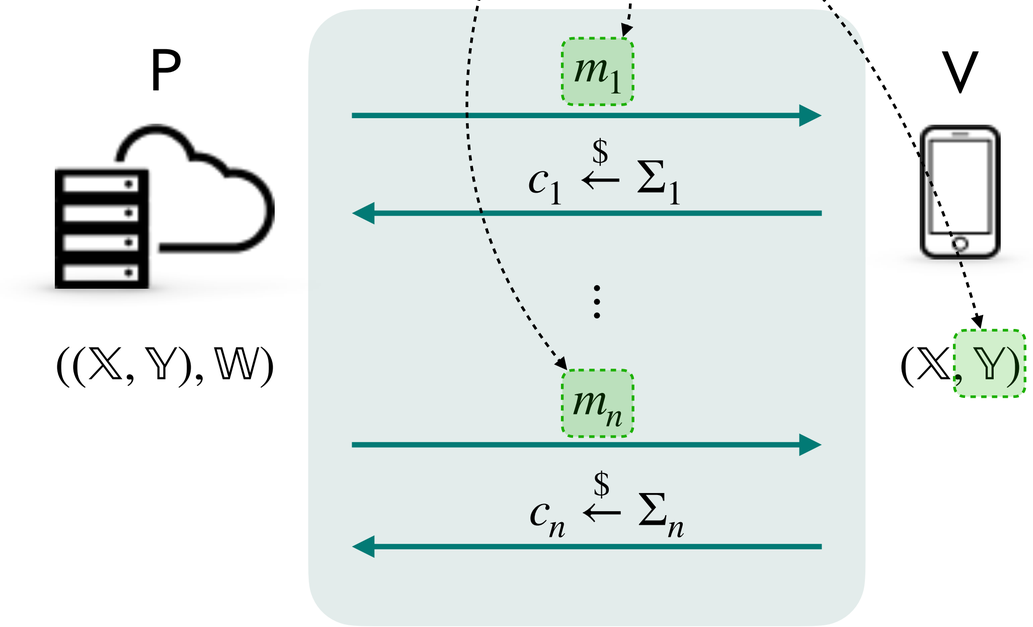
Interactive Argument

Anatomy of a SNARK

Oracle Interface (available to V):

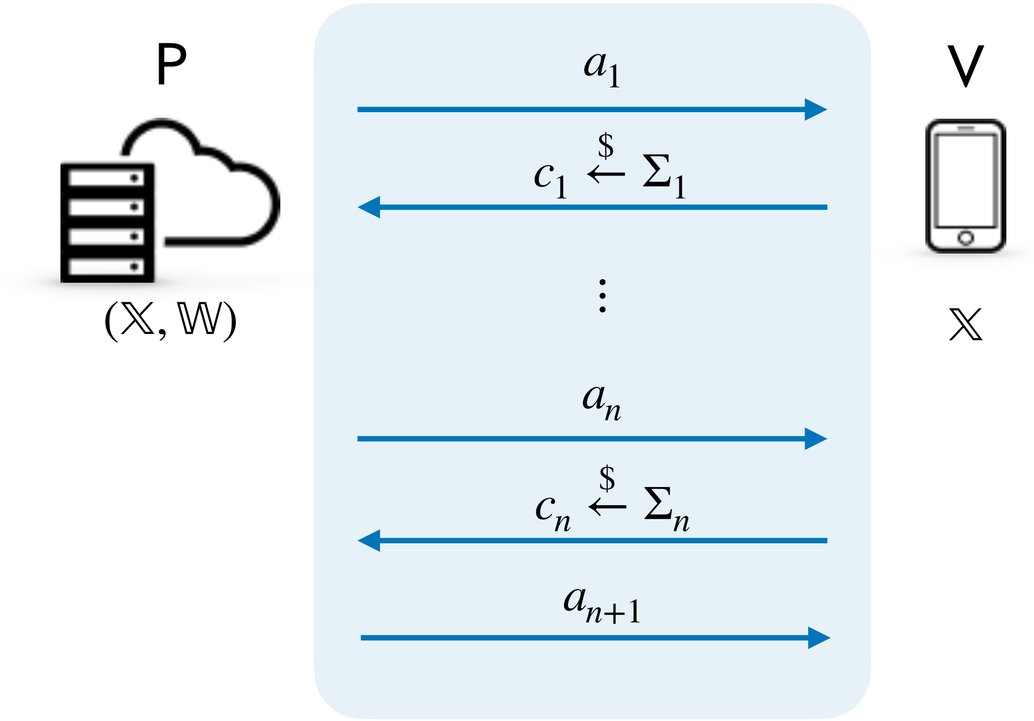
$(Q, R, M \to Q \to R)$

Vector query,
polynomial query, etc.



(interactive)

BCS



Interactive Oracle Proof

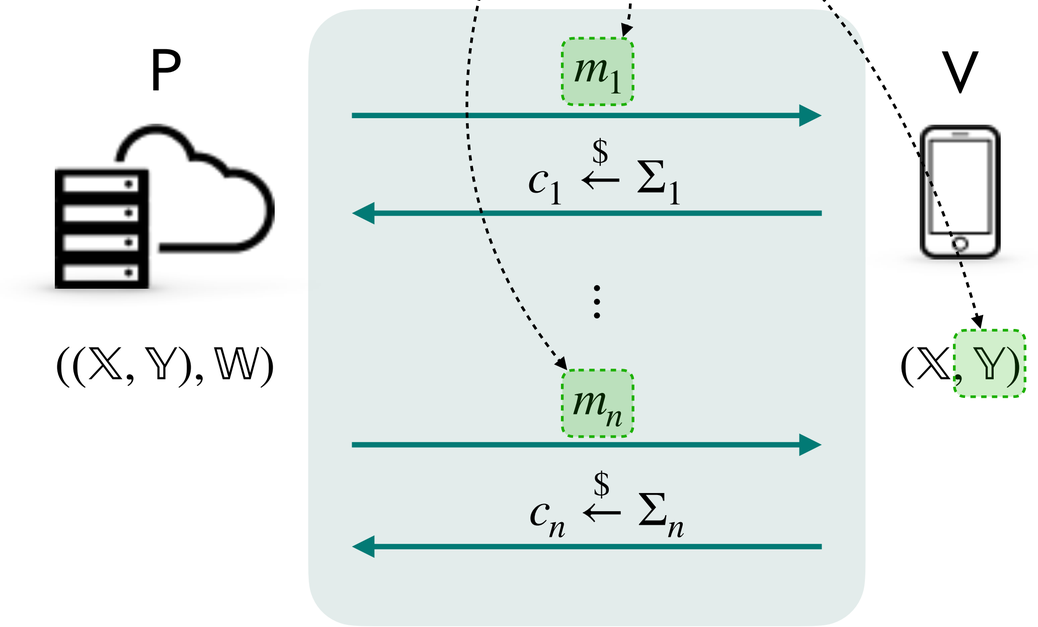
Interactive Argument

Anatomy of a SNARK

Oracle Interface (available to V):

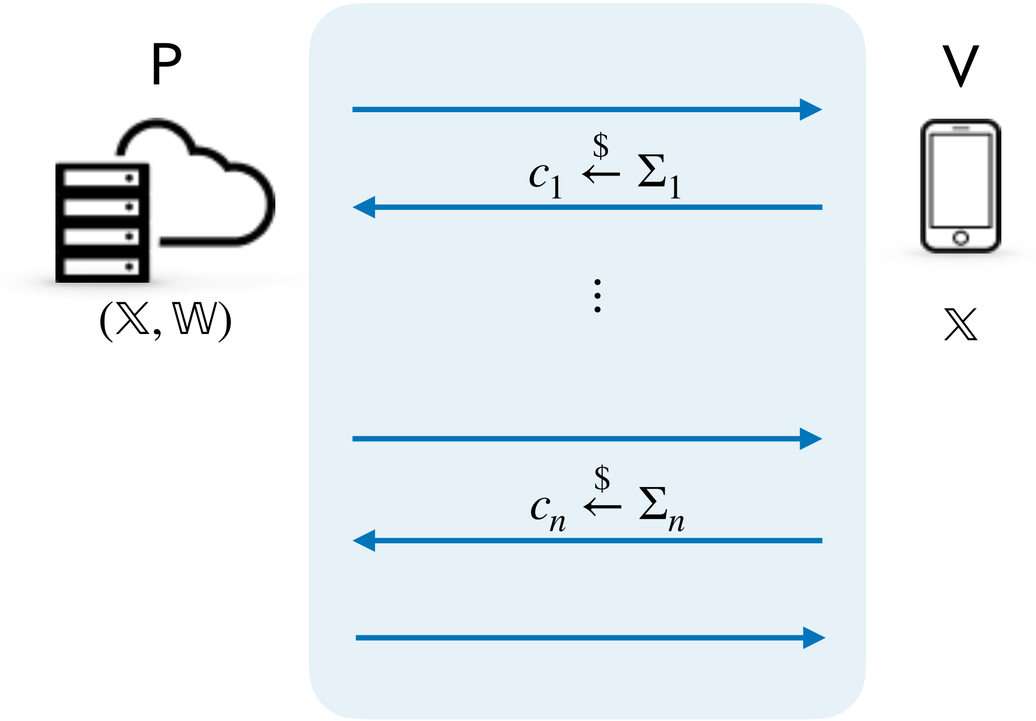
$(Q, R, M \to Q \to R)$

Vector query,
polynomial query, etc.



(interactive)

BCS



Interactive Oracle Proof

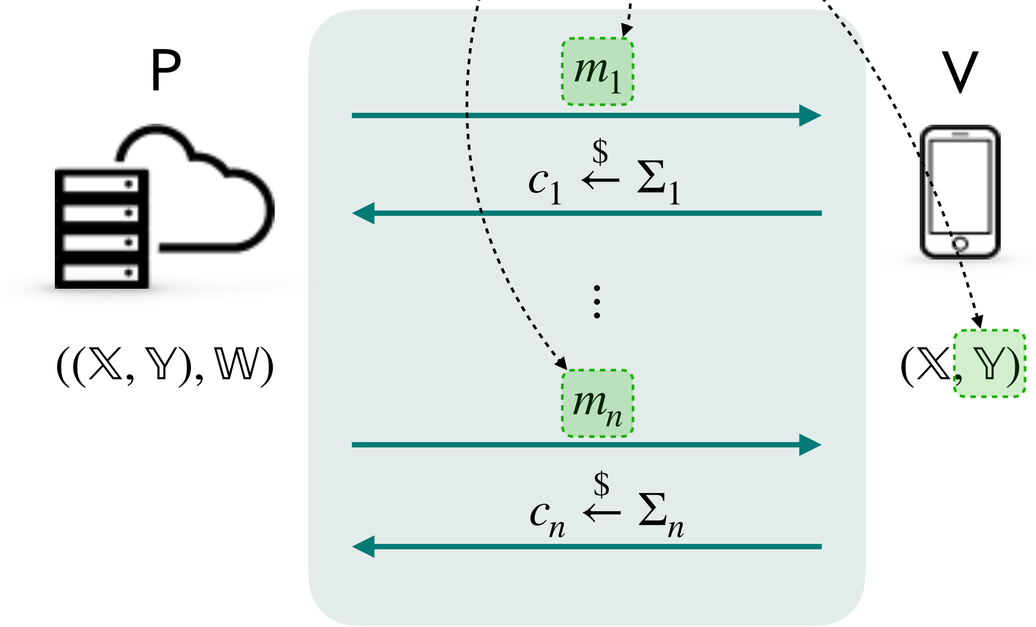
Interactive Argument

Anatomy of a SNARK

Oracle Interface (available to V):

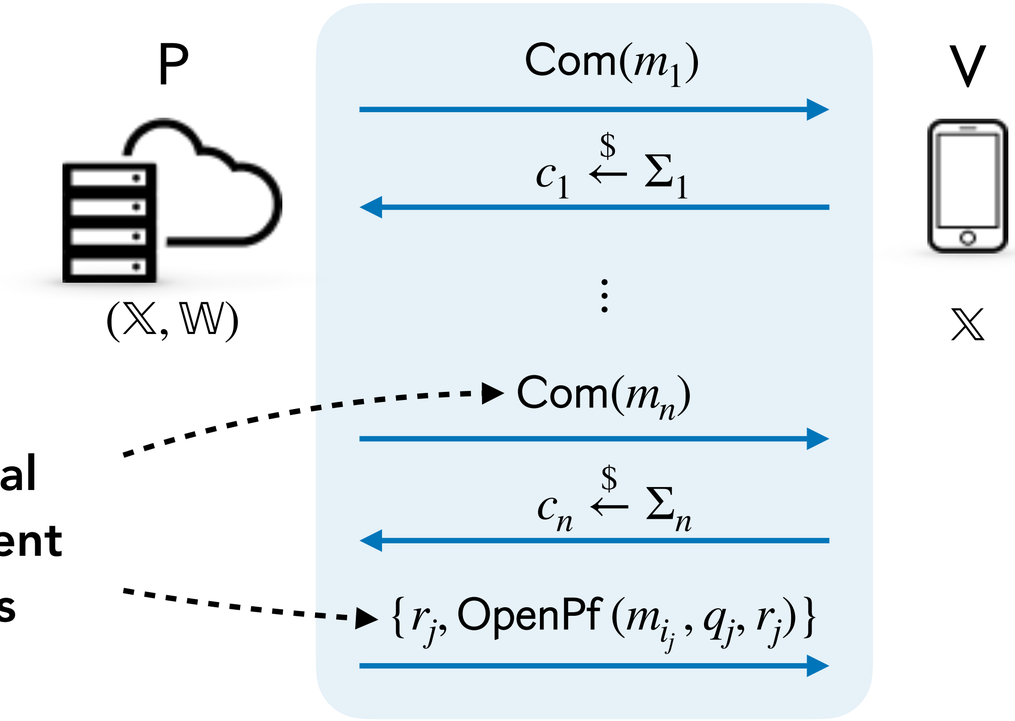
$(Q, R, M \to Q \to R)$

Vector query,
polynomial query, etc.



(interactive)

BCS



Functional commitment schemes

Interactive Oracle Proof

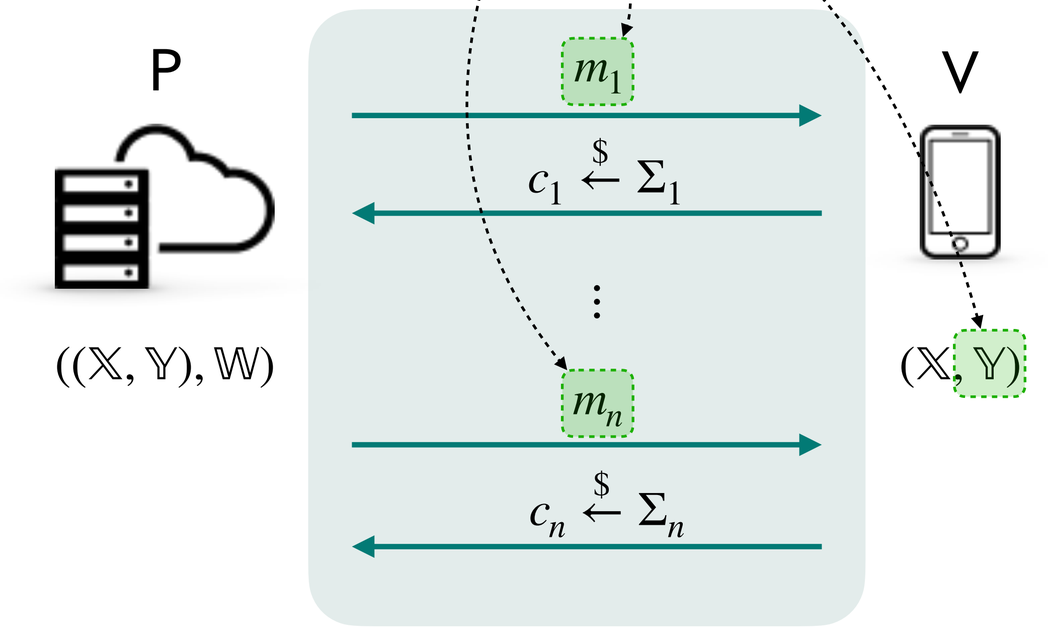
Interactive Argument

Interactive Oracle Reductions (IORs)

Oracle Interface (available to V):

$(Q, R, M \to Q \to R)$

Vector query,
polynomial query, etc.



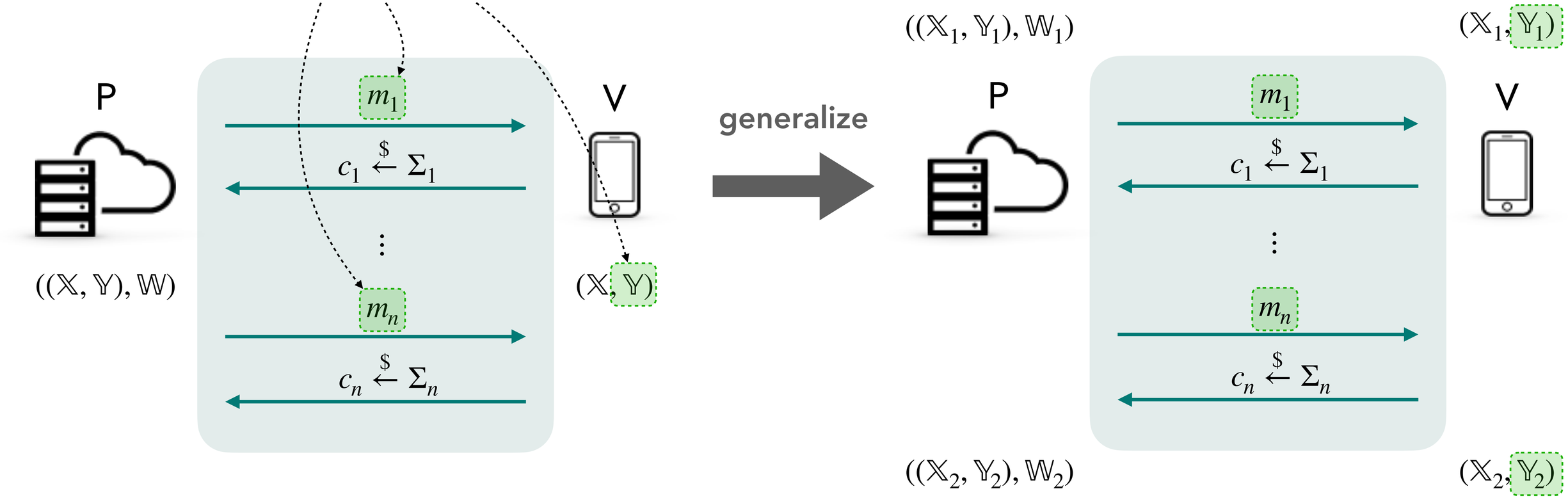
Interactive Oracle Proof

Interactive Oracle Reductions (IORs)

Oracle Interface (available to V):

$(Q, R, M \to Q \to R)$

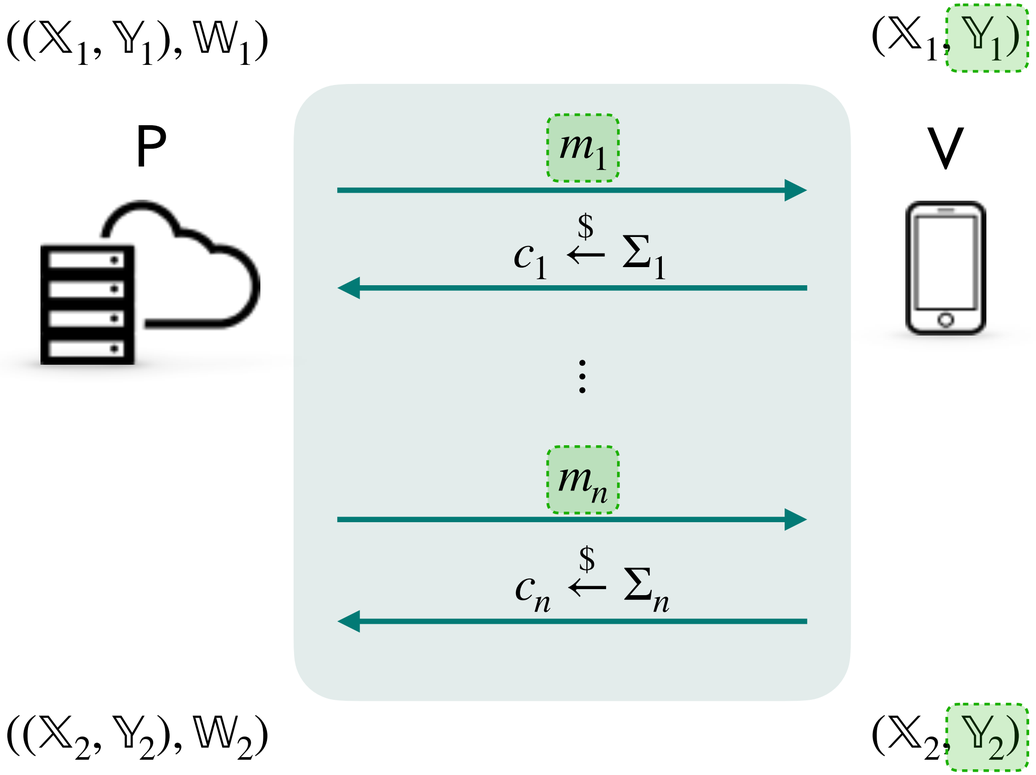
Vector query,
polynomial query, etc.



Interactive Oracle Proof

$\mathbb{Y}_2$ <u>implicitly</u> defined in terms of $\mathbb{X}_1, (c_1, \ldots, c_n)$,
and oracle access to $\mathbb{Y}_1, (m_1, \ldots, m_n)$

Properties of IORs



$\mathbb{Y}_2$ <u>implicitly</u> defined in terms of $\mathbb{X}_1, (c_1, \ldots, c_n)$,
and oracle access to $\mathbb{Y}_1, (m_1, \ldots, m_n)$

Properties of IORs

Completeness: if $((\mathbb{X}_1, \mathbb{Y}_1), \mathbb{W}_1) \in \mathscr{R}_1$

then $\langle P, V \rangle(\ldots) \to ((\mathbb{X}_2, \mathbb{Y}_2), \mathbb{W}_2) \in \mathscr{R}_2$

Knowledge Soundness:

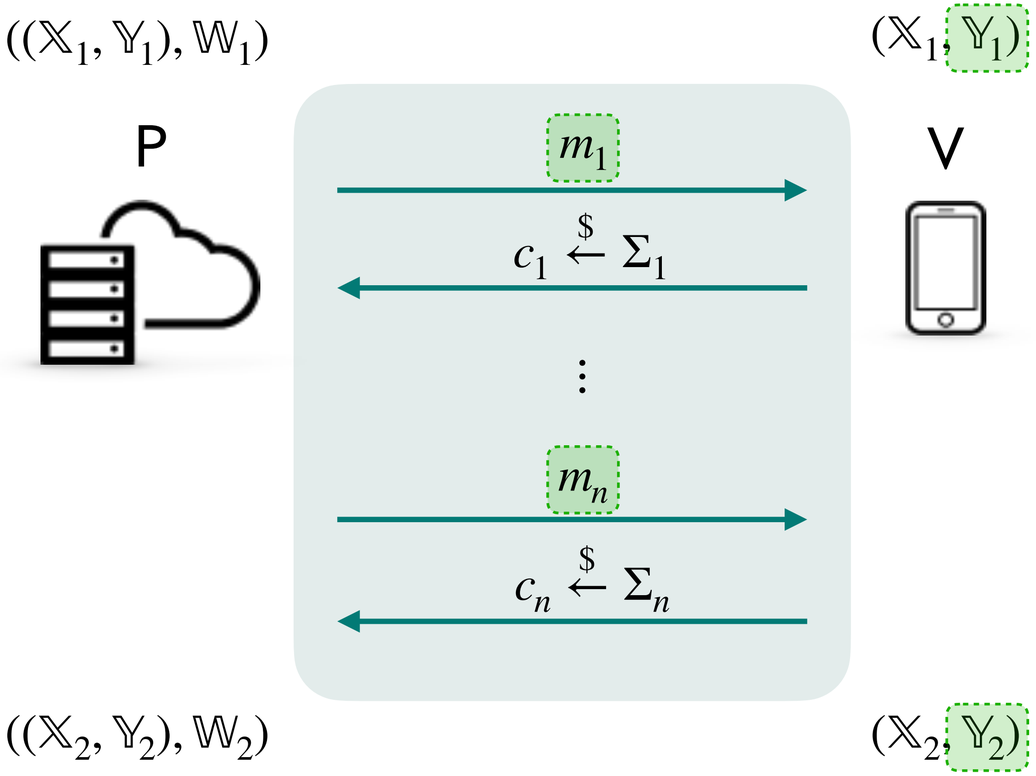
$\exists$ extractor $\mathsf{E}(\mathbb{W}_2, \ldots) \to \mathbb{W}_1$

such that for all $\mathsf{P}^\star$, if

$\langle \mathsf{P}^\star, \mathsf{V} \rangle(\ldots) \to ((\mathbb{X}_2, \mathbb{Y}_2), \mathbb{W}_2) \in \mathscr{R}_2$, then

$(\mathbb{W}_1, (\mathbb{X}_1, \mathbb{Y}_1)) \in \mathscr{R}_1$, except with prob. ϵ

(Honest-Verifier) Zero-Knowledge



$\mathbb{Y}_2$ <u>implicitly</u> defined in terms of $\mathbb{X}_1, (c_1, \ldots, c_n)$, and oracle access to $\mathbb{Y}_1, (m_1, \ldots, m_n)$

Properties of IORs

Completeness: if $((\mathbb{X}_1, \mathbb{Y}_1), \mathbb{W}_1) \in \mathscr{R}_1$

then $\langle P, V \rangle(\ldots) \to ((\mathbb{X}_2, \mathbb{Y}_2), \mathbb{W}_2) \in \mathscr{R}_2$

Knowledge Soundness:

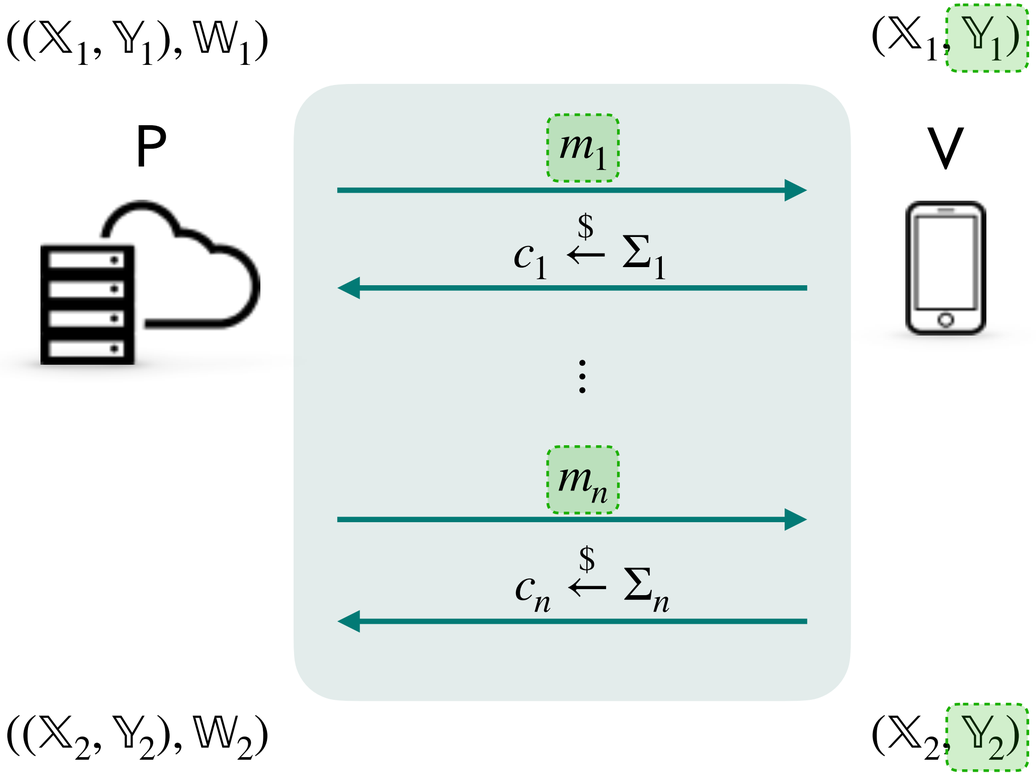
needs stronger notions!
(i.e. state-restoration, round-by-round)

$\exists$ extractor $\mathsf{E}(\mathbb{W}_2, \ldots) \to \mathbb{W}_1$

such that for all $\mathsf{P}^\star$, if

$\langle \mathsf{P}^\star, \mathsf{V} \rangle(\ldots) \to ((\mathbb{X}_2, \mathbb{Y}_2), \mathbb{W}_2) \in \mathscr{R}_2$, then

$(\mathbb{W}_1, (\mathbb{X}_1, \mathbb{Y}_1)) \in \mathscr{R}_1$, except with prob. ϵ



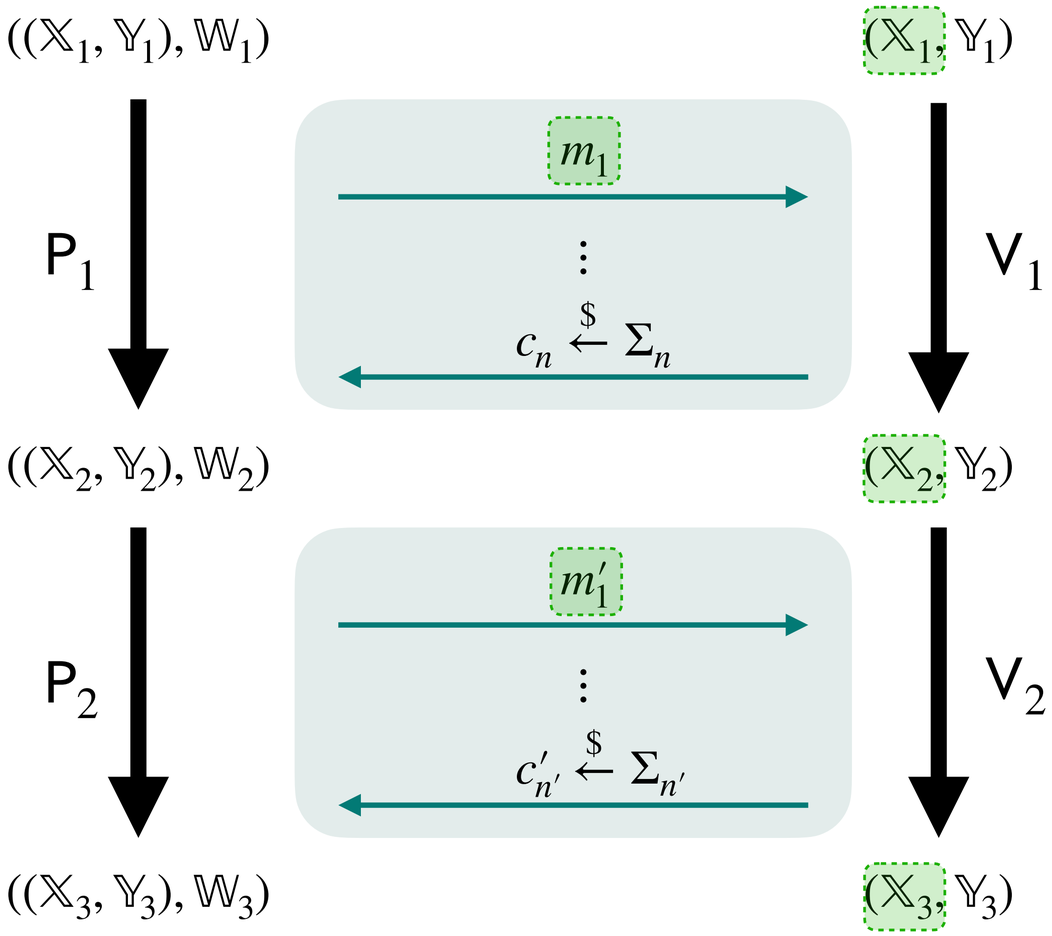
(Honest-Verifier) Zero-Knowledge

$\mathbb{Y}_2$ implicitly defined in terms of $\mathbb{X}_1, (c_1, \ldots, c_n)$, and oracle access to $\mathbb{Y}_1, (m_1, \ldots, m_n)$

Why IORs?

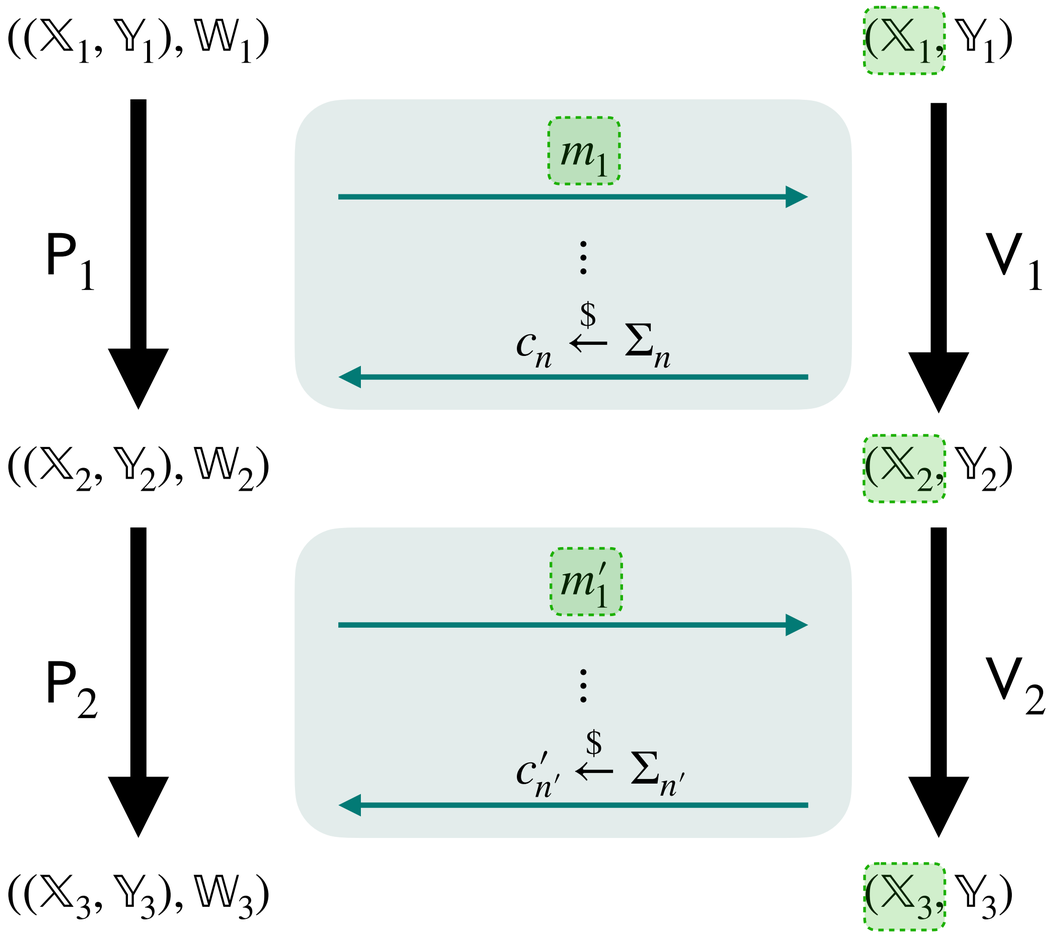
Why IORs?

Sequential Composition

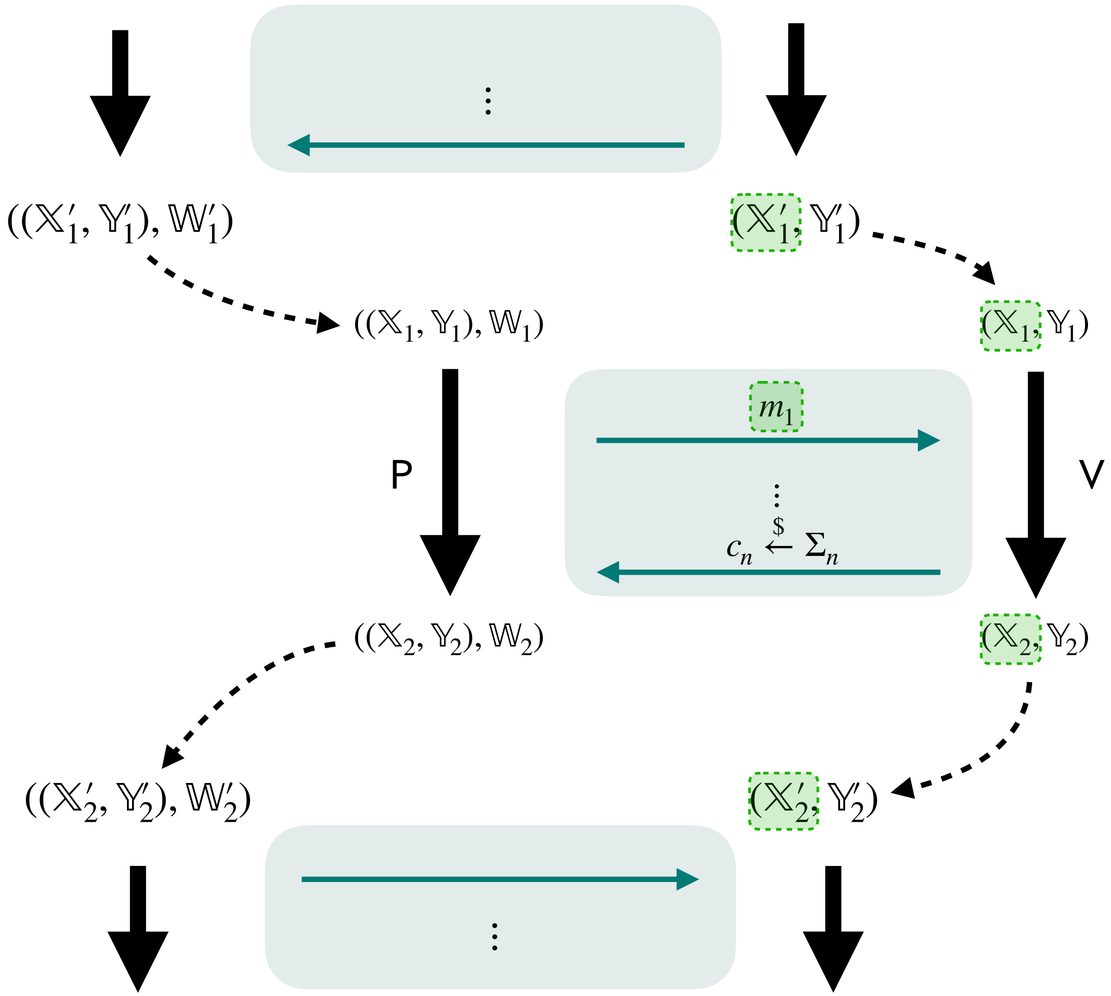


Why IORs?

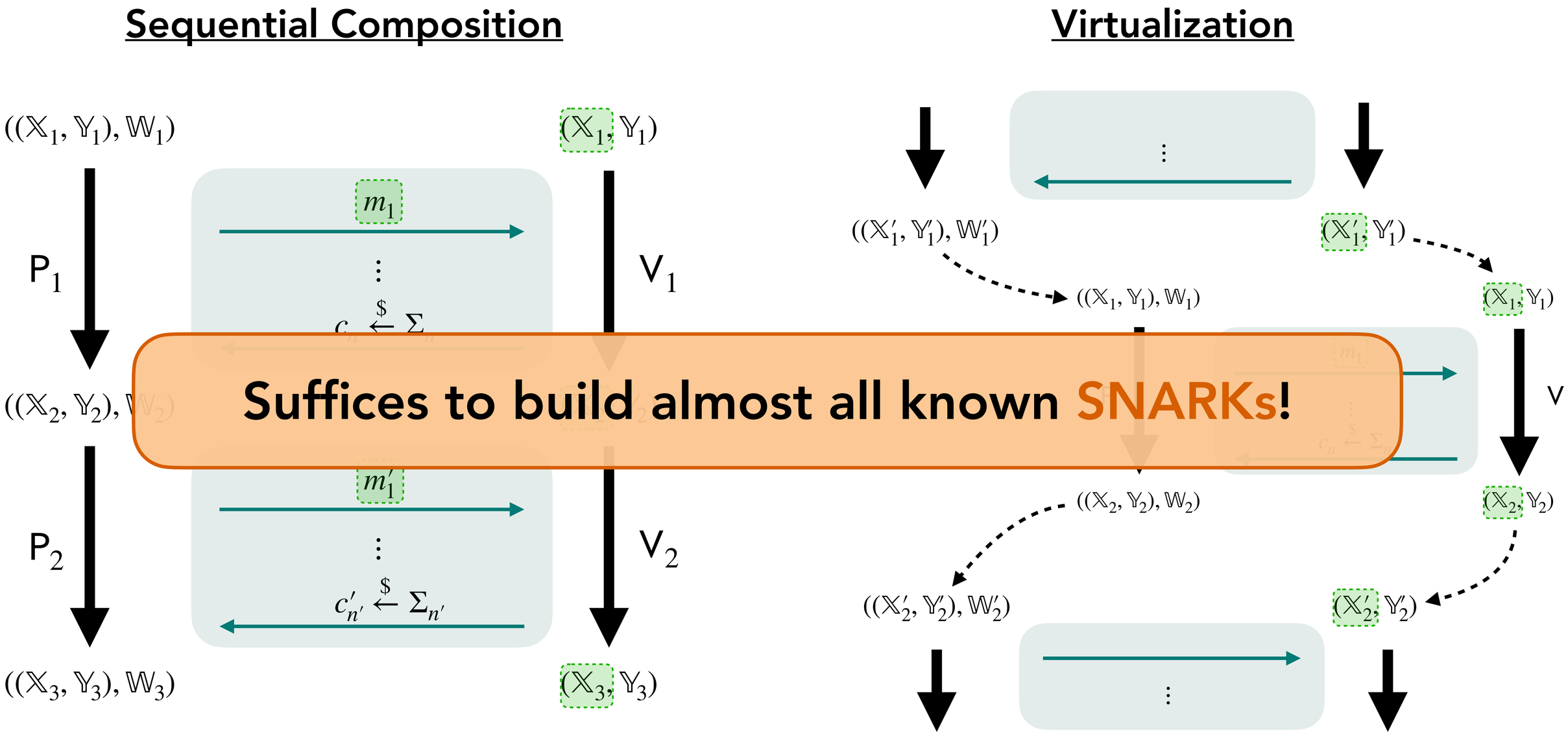
Sequential Composition



Virtualization



Why IORs?

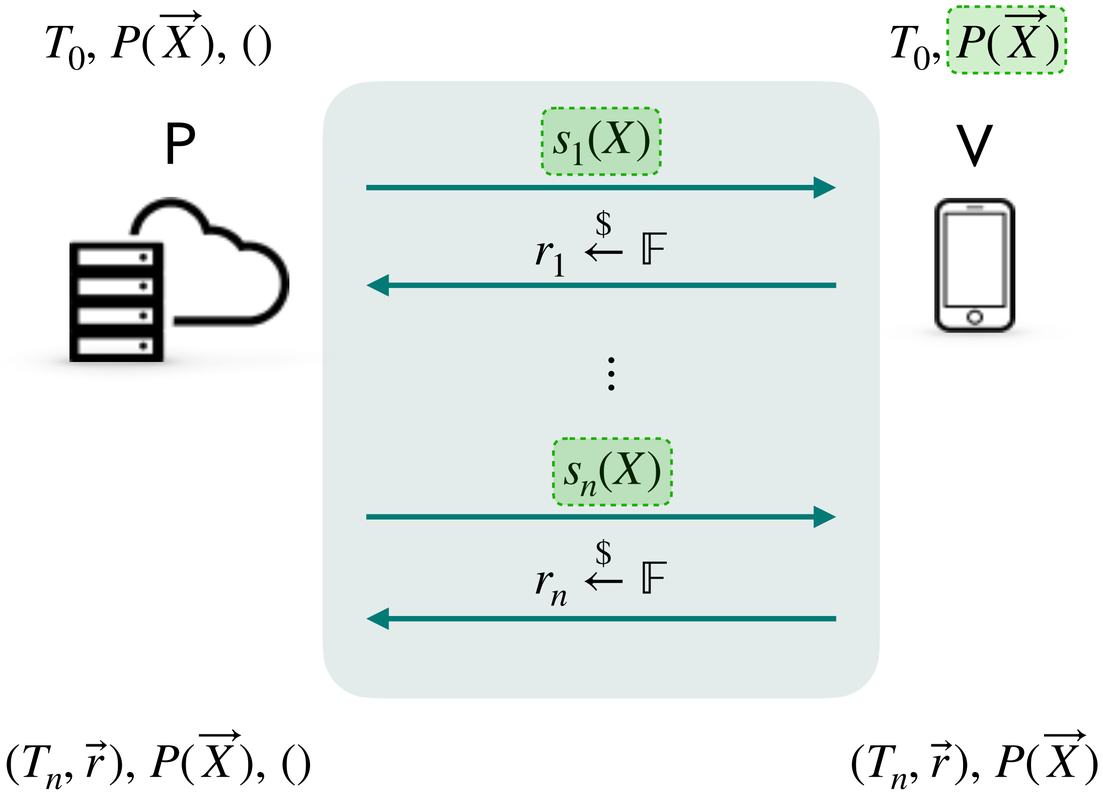


Suffices to build almost all known SNARKs!

Formalizing Sum-Check in ArkLib

Formalizing Sum-Check in ArkLib

Input Relation: $\sum_{x \in \{0,1\}^n} P(x) = T_0$



Output Relation:

$P(r_1, \ldots, r_n) = T_n \ \wedge$ V hasn't rejected

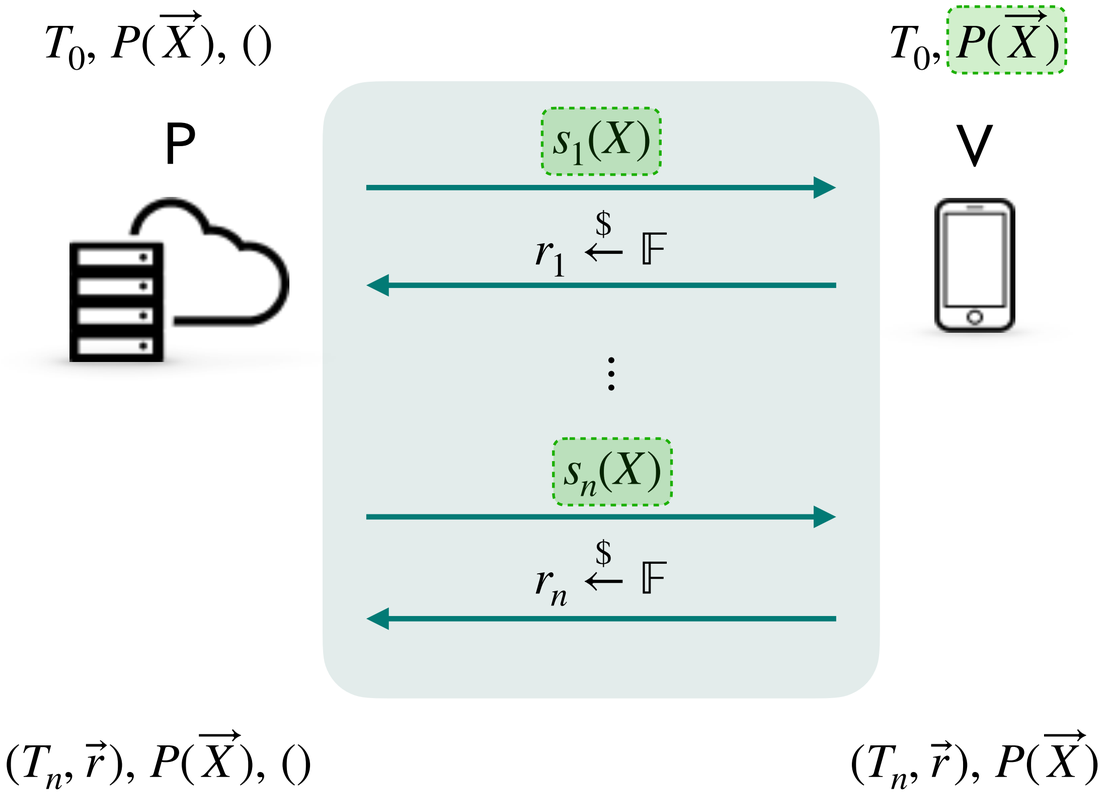
Formalizing Sum-Check in ArkLib

For round $i = 1, \ldots, n$:

1. P sends $s_i(X) = \sum_{x' \in \{0,1\}^{n-i-1}} P(r_1, \ldots, r_{i-1}, X, x')$
2. V sends $r_i \xleftarrow{\$} \mathbb{F}$
3. V rejects if $s_i(0) + s_i(1) \neq T_{i-1}$,

   otherwise outputs $T_i := s_i(r_i)$.

Input Relation: $\sum_{x \in \{0,1\}^n} P(x) = T_0$



Output Relation:

$P(r_1, \ldots, r_n) = T_n \ \wedge$ V hasn't rejected

Formalizing Sum-Check in ArkLib

Sequential Composition

$\Longrightarrow$ focus on first round

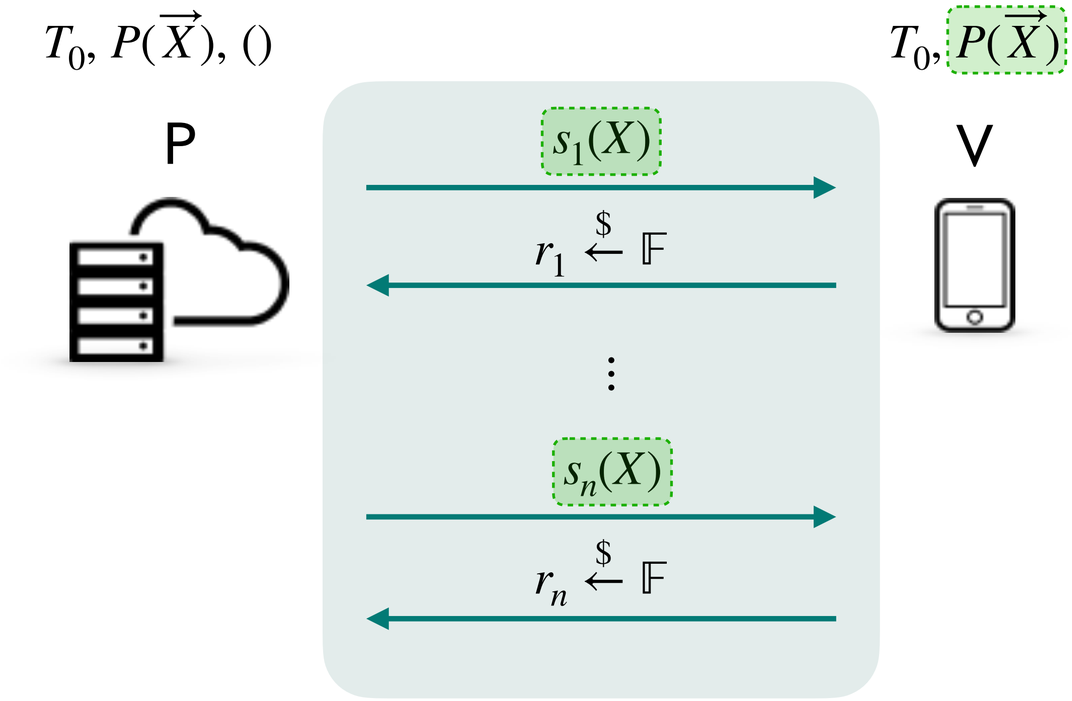
For round $i = 1, \ldots, n$:

1. P sends $s_i(X) = \sum_{x' \in \{0,1\}^{n-i-1}} P(r_1, \ldots, r_{i-1}, X, x')$
2. V sends $r_i \xleftarrow{\$} \mathbb{F}$
3. V rejects if $s_i(0) + s_i(1) \neq T_{i-1}$,

   otherwise outputs $T_i := s_i(r_i)$.

Input Relation: $\sum_{x \in \{0,1\}^n} P(x) = T_0$

$T_0, P(\vec{X}), ()$ $\qquad$ $T_0, P(\vec{X})$



$(T_n, \vec{r}), P(\vec{X}), ()$ $\qquad$ $(T_n, \vec{r}), P(\vec{X})$

Output Relation:

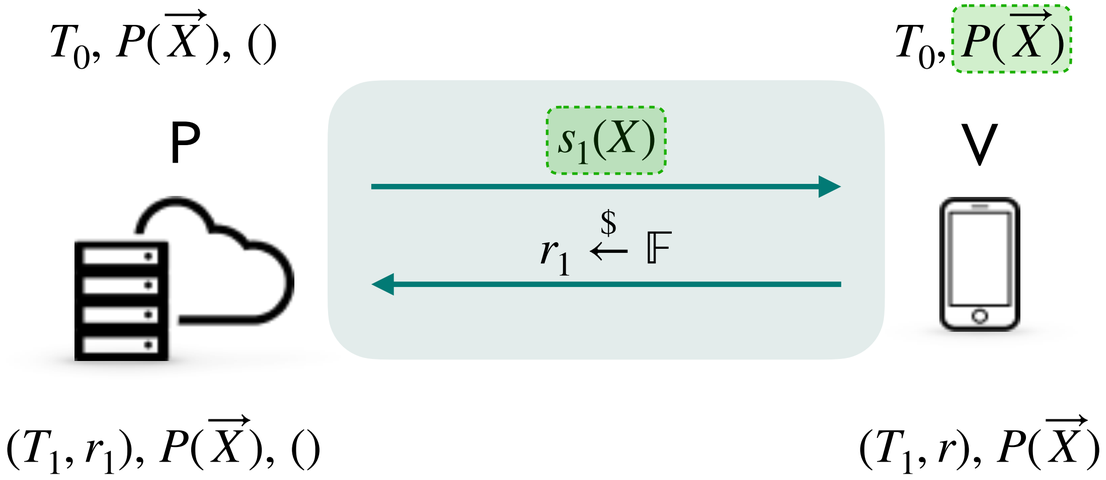
$P(r_1, \ldots, r_n) = T_n \ \wedge$ V hasn't rejected

Formalizing Sum-Check in ArkLib

Sequential Composition

$\Longrightarrow$ focus on first round

Input Relation: $\sum_{x \in \{0,1\}^n} P(x) = T_0$



Output Relation:

$$\sum_{x' \in \{0,1\}^{n-1}} P(r_1, x') = T_1 \ \wedge \ \text{V hasn't rejected}$$

Formalizing Sum-Check in ArkLib

Sequential Composition

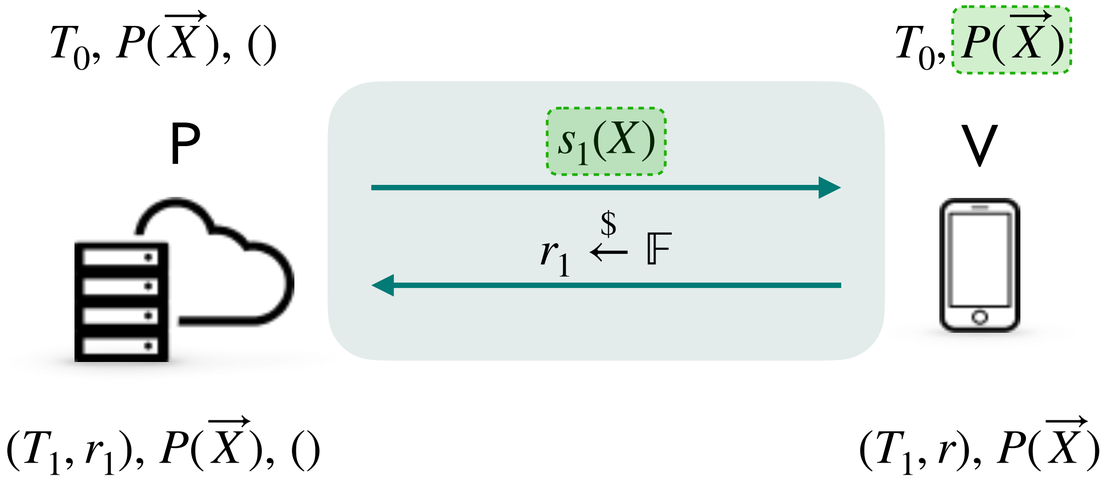
$\Longrightarrow$ focus on first round

In round $i = 1$:

1. P sends $s_1(X) = \sum_{x' \in \{0,1\}^{n-1}} P(X, x')$
2. V sends $r_1 \xleftarrow{\$} \mathbb{F}$
3. V rejects if $s_1(0) + s_1(1) \neq T_0$,

   otherwise outputs $T_1 := s_1(r_1)$.

Input Relation: $\sum_{x \in \{0,1\}^n} P(x) = T_0$

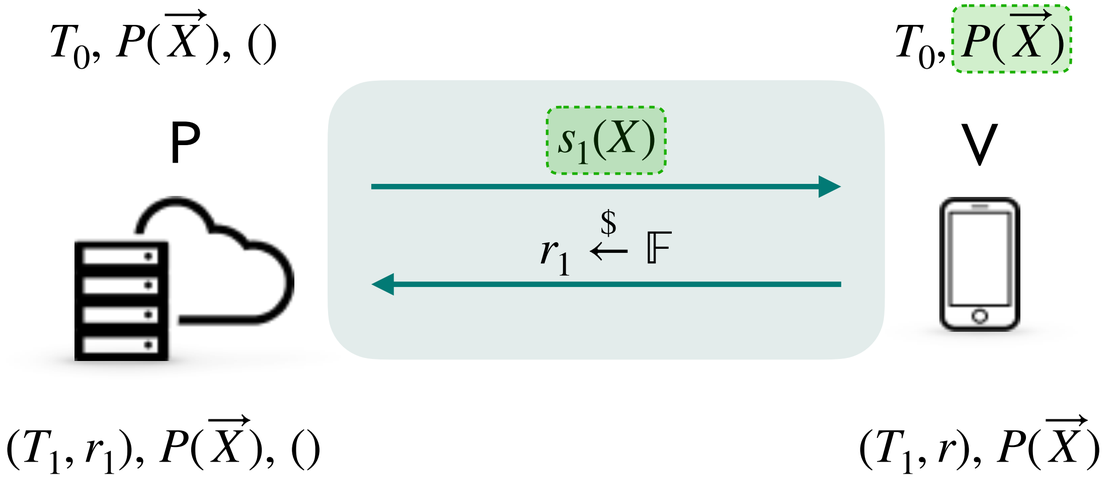


Output Relation:

$$\sum_{x' \in \{0,1\}^{n-1}} P(r_1, x') = T_1 \ \wedge \ \text{V hasn't rejected}$$

Formalizing Sum-Check in ArkLib

Input Relation: $\sum_{x \in \{0,1\}^n} P(x) = T_0$



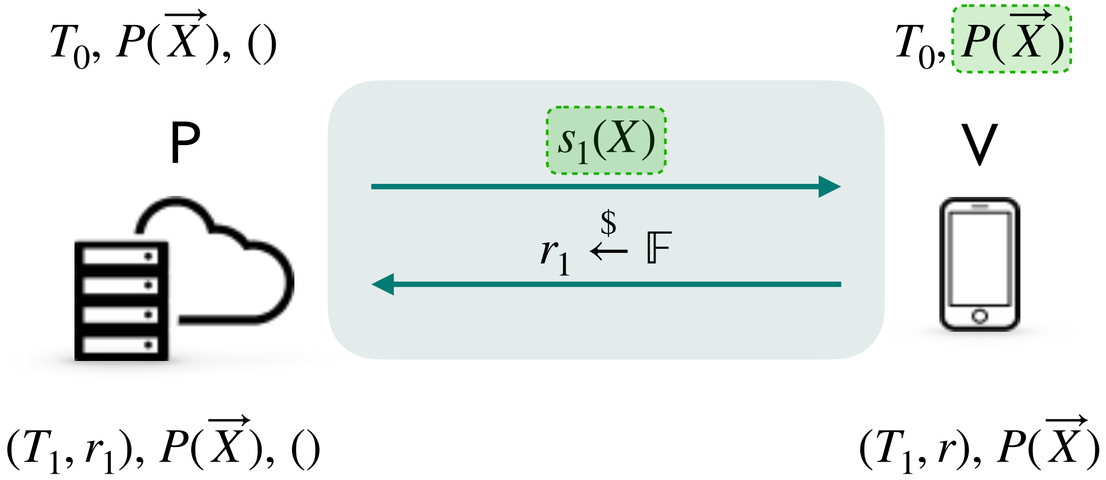
Output Relation:

$$\sum_{x' \in \{0,1\}^{n-1}} P(r_1, x') = T_1 \ \wedge \ \text{V hasn't rejected}$$

Formalizing Sum-Check in ArkLib

1. Define the protocol format

Input Relation: $\sum_{x \in \{0,1\}^n} P(x) = T_0$



Output Relation:

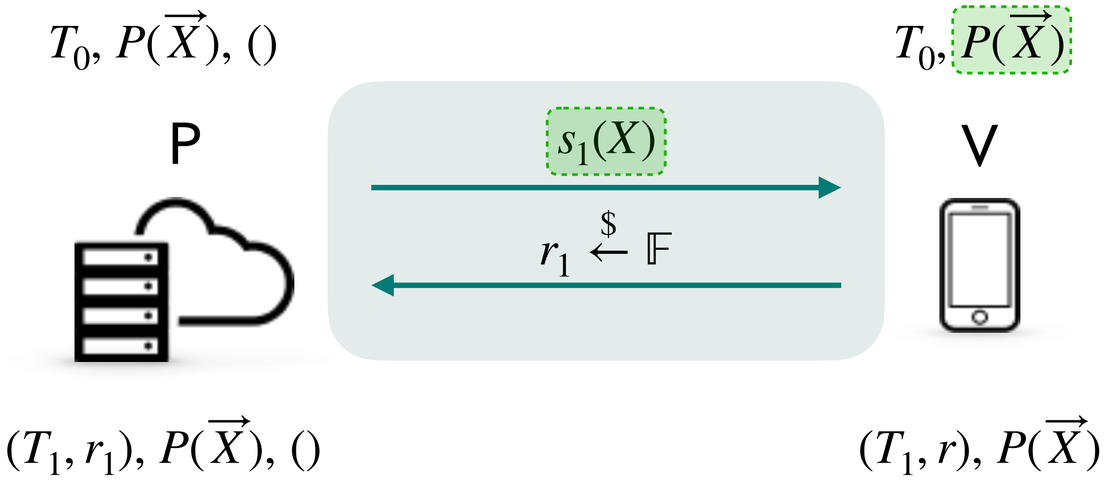
$$\sum_{x' \in \{0,1\}^{n-1}} P(r_1, x') = T_1 \ \wedge \ \text{V hasn't rejected}$$

Formalizing Sum-Check in ArkLib

1. Define the protocol format

```
inductive Direction where
  | P_to_V | V_to_P

def ProtocolSpec := List (Direction × Type)

variable (F : Type) [Field F] (deg : ℕ)

def scRoundSpec : ProtocolSpec :=
  [ (.P_to_V, F⦃≤ deg⦄[X]),
    (.V_to_P, F) ]
```

Input Relation: $\sum_{x \in \{0,1\}^n} P(x) = T_0$



Output Relation:

$$\sum_{x' \in \{0,1\}^{n-1}} P(r_1, x') = T_1 \ \wedge \ \text{V hasn't rejected}$$

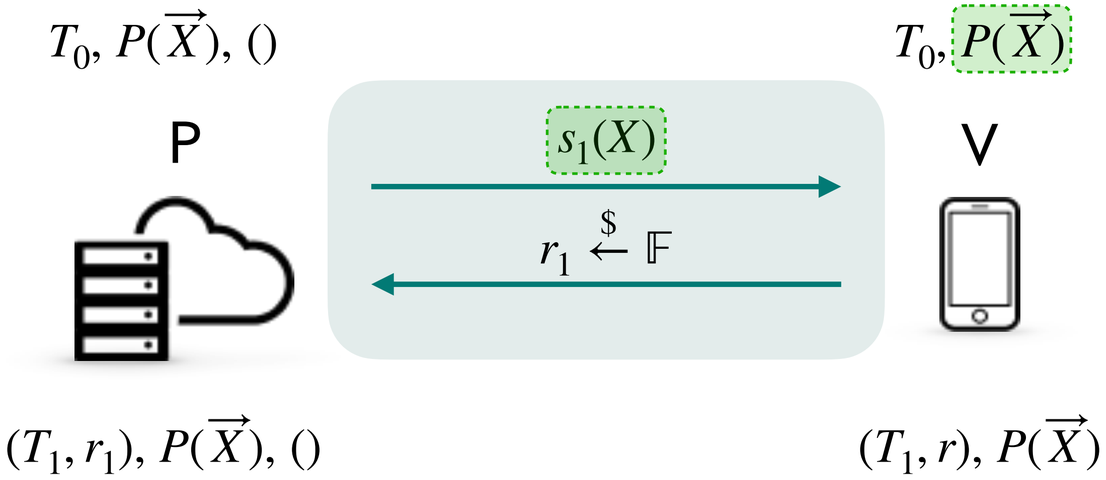
Formalizing Sum-Check in ArkLib

1. Define the protocol format

```
inductive Direction where
  | P_to_V | V_to_P

def ProtocolSpec := List (Direction × Type)

variable (F : Type) [Field F] (deg : ℕ)

def scRoundSpec : ProtocolSpec :=
  [ (.P_to_V, F⦃≤ deg⦄[X]),
    (.V_to_P, F) ]
```

```
def Transcript : ProtocolSpec → Type
  | [] => Unit
  | (_, T) :: rest => T × Transcript rest

-- returns (F⦃≤ deg⦄[X] × F × Unit)
#eval Transcript scRoundSpec
```

Input Relation: $\sum_{x \in \{0,1\}^n} P(x) = T_0$



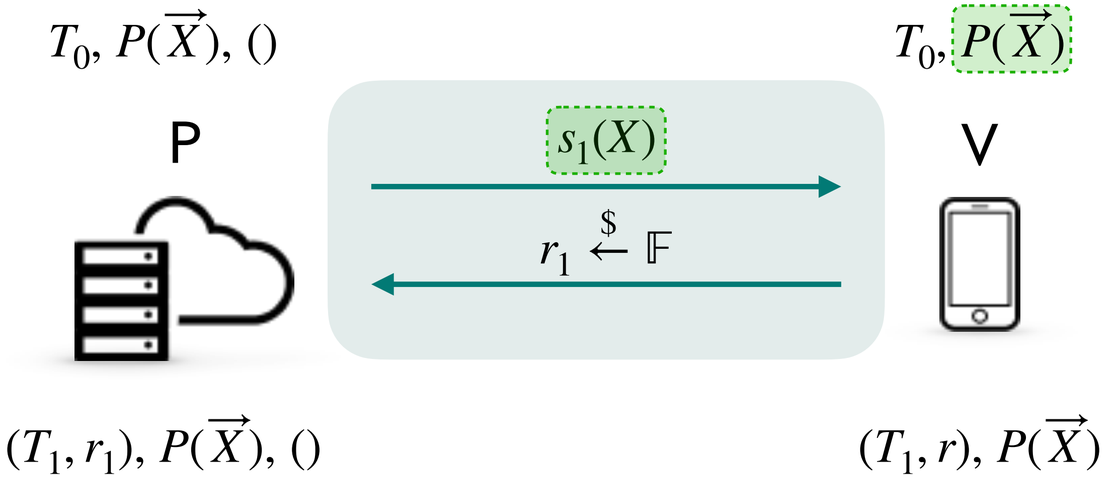
Output Relation:

$$\sum_{x' \in \{0,1\}^{n-1}} P(r_1, x') = T_1 \ \wedge \ \text{V hasn't rejected}$$

Formalizing Sum-Check in ArkLib

2. Define the prover & verifier

Input Relation: $\sum_{x \in \{0,1\}^n} P(x) = T_0$



Output Relation:

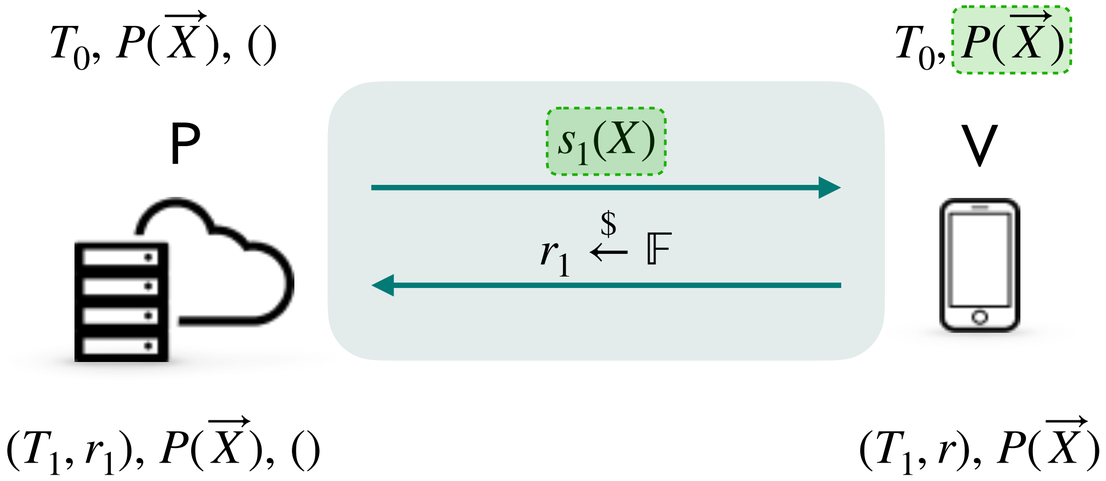
$$\sum_{x' \in \{0,1\}^{n-1}} P(r_1, x') = T_1 \ \wedge \ \text{V hasn't rejected}$$

Formalizing Sum-Check in ArkLib

2. Define the prover & verifier

```
def Prover (Output : Type) : ProtocolSpec → Type
  | [] => Output
  | (.P_to_V, Msg) :: rest => Msg × Prover Output rest
  | (.V_to_P, Chal) :: rest => Chal → Prover Output rest

def scRoundProver (T₀ : F) (P : F⦃≤ d⦄[X_⦃n⦄]) :
    Prover (F × F × F⦃≤ d⦄[X_⦃n⦄]) (scRoundSpec F d) :=
  let sPoly := ∑ b ∈ ⦃0, 1⦄^⦃n-1⦄, P (X, b)
  (sPoly, fun r => (sPoly (r), r, P))
```

Input Relation: $\sum_{x \in \{0,1\}^n} P(x) = T_0$



Output Relation:

$$\sum_{x' \in \{0,1\}^{n-1}} P(r_1, x') = T_1 \ \wedge \ \text{V hasn't rejected}$$

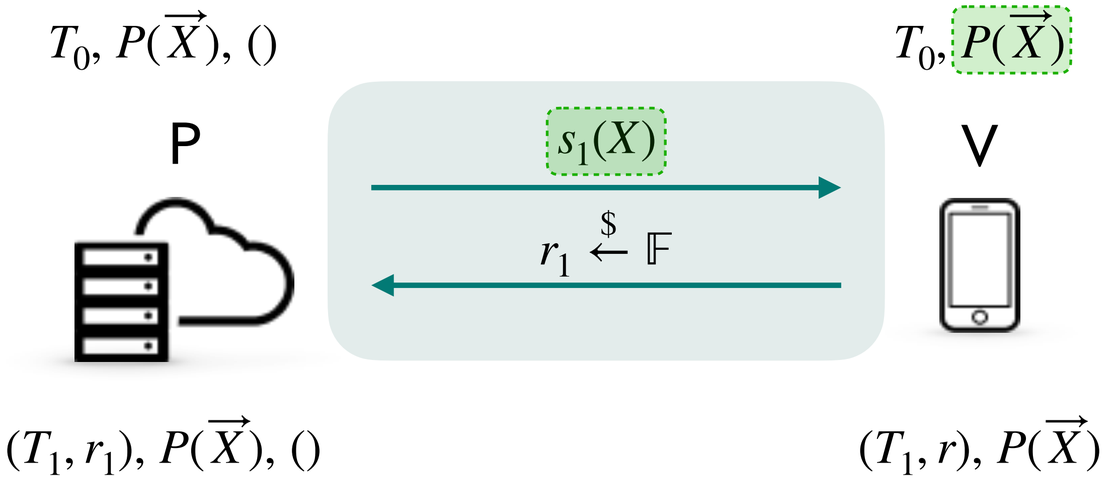
Formalizing Sum-Check in ArkLib

2. Define the prover & verifier

```
def Prover (Output : Type) : ProtocolSpec → Type
  | [] => Output
  | (.P_to_V, Msg) :: rest => Msg × Prover Output rest
  | (.V_to_P, Chal) :: rest => Chal → Prover Output rest

def scRoundProver (T₀ : F) (P : F⦃≤ d⦄[X_⦃n⦄]) :
    Prover (F × F × F⦃≤ d⦄[X_⦃n⦄]) (scRoundSpec F d) :=
  let sPoly := ∑ b ∈ ⦃0, 1⦄^⦃n-1⦄, P (X, b)
  (sPoly, fun r => (sPoly (r), r, P))
```

```
def Verifier (Input Output : Type) (pSpec : ProtocolSpec) :=
  Input → Transcript pSpec → Option Output

def scRoundVerifier : Verifier (F × F⦃≤ d⦄[X_⦃n⦄])
    (F × F × F⦃≤ d⦄[X_⦃n⦄]) (scRoundSpec F d) :=
  fun (T₀, P) (s, r, ()) =>
    if s (0) + s (1) = T₀
      then some (s (r), r, P) else none
```

Input Relation: $\sum_{x \in \{0,1\}^n} P(x) = T_0$



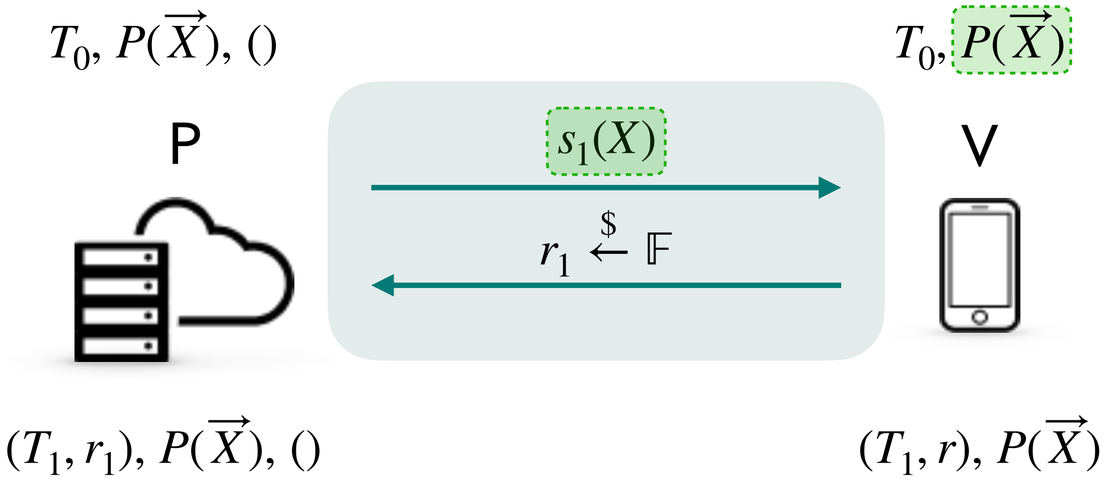
Output Relation:

$$\sum_{x' \in \{0,1\}^{n-1}} P(r_1, x') = T_1 \ \wedge \ \text{V hasn't rejected}$$

Formalizing Sum-Check in ArkLib

3. State and prove security

Input Relation: $\sum_{x \in \{0,1\}^n} P(x) = T_0$



Output Relation:

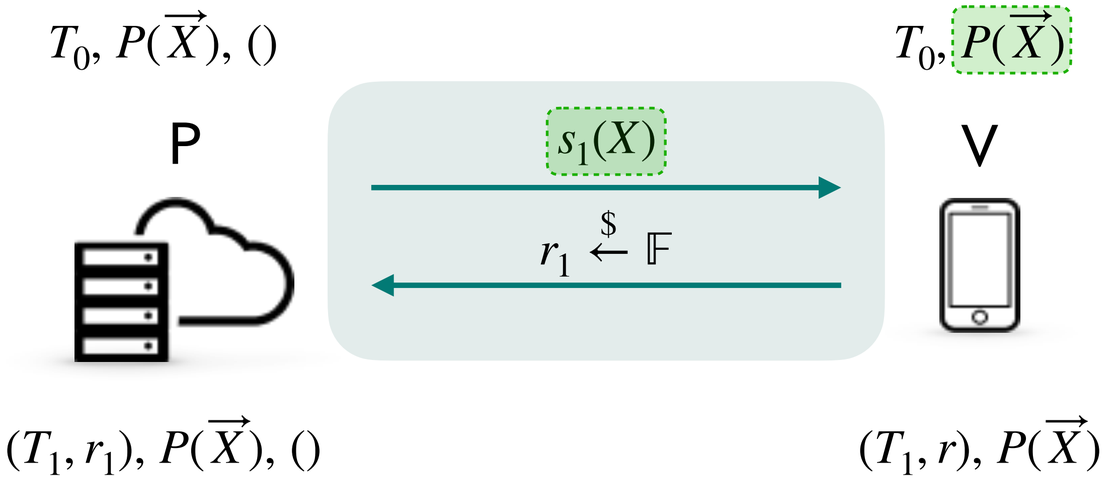
$$\sum_{x' \in \{0,1\}^{n-1}} P(r_1, x') = T_1 \ \wedge \ \text{V hasn't rejected}$$

Formalizing Sum-Check in ArkLib

3. State and prove security

```
def Verifier.soundness (langIn : Set StmtIn)
    (langOut : Set StmtOut)
    (verifier : Verifier StmtIn StmtOut pSpec)
    (ε : R≥0) : Prop :=
  ∀ stmtIn ∉ langIn, ∀ prover,
    Pr[ fun result =>
          result = some stmtOut ∧ stmtOut ∈ langOut
      | do return ⟪prover, verifier⟫ stmtIn ] ≤ ε

def scInputLang : Set (F × F⦃≤ d⦄[X_⦃n⦄]) :=
  { (T₀, P) | T₀ = ∑ x ∈ ⦃0,1⦄^⦃n⦄, P (x) }

def scOutputLang : Set (F × F × F⦃≤ d⦄[X_⦃n⦄]) :=
  { (T₁, r, P) | T₁ = ∑ x' ∈ ⦃0,1⦄^⦃n-1⦄, P (r, x') }

theorem scRound_sound :
    (scRoundVerifier F d n).soundness
      (scInputLang F d n) (scOutputLang F d n)
        (d / #(F)) := sorry
```

Input Relation: $\sum_{x \in \{0,1\}^n} P(x) = T_0$

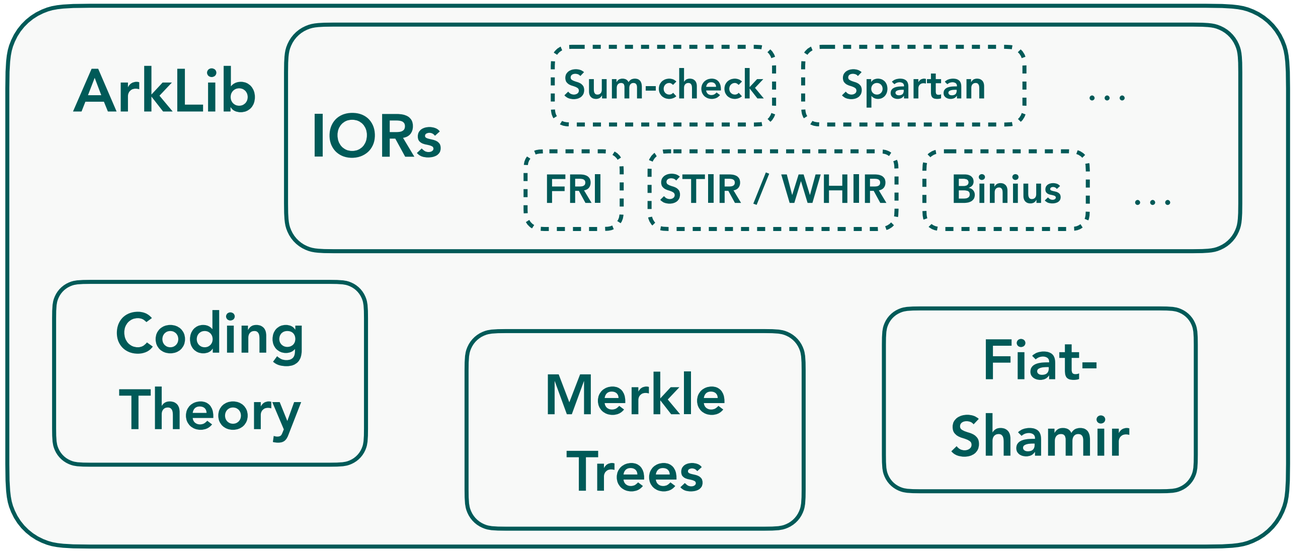


Output Relation:

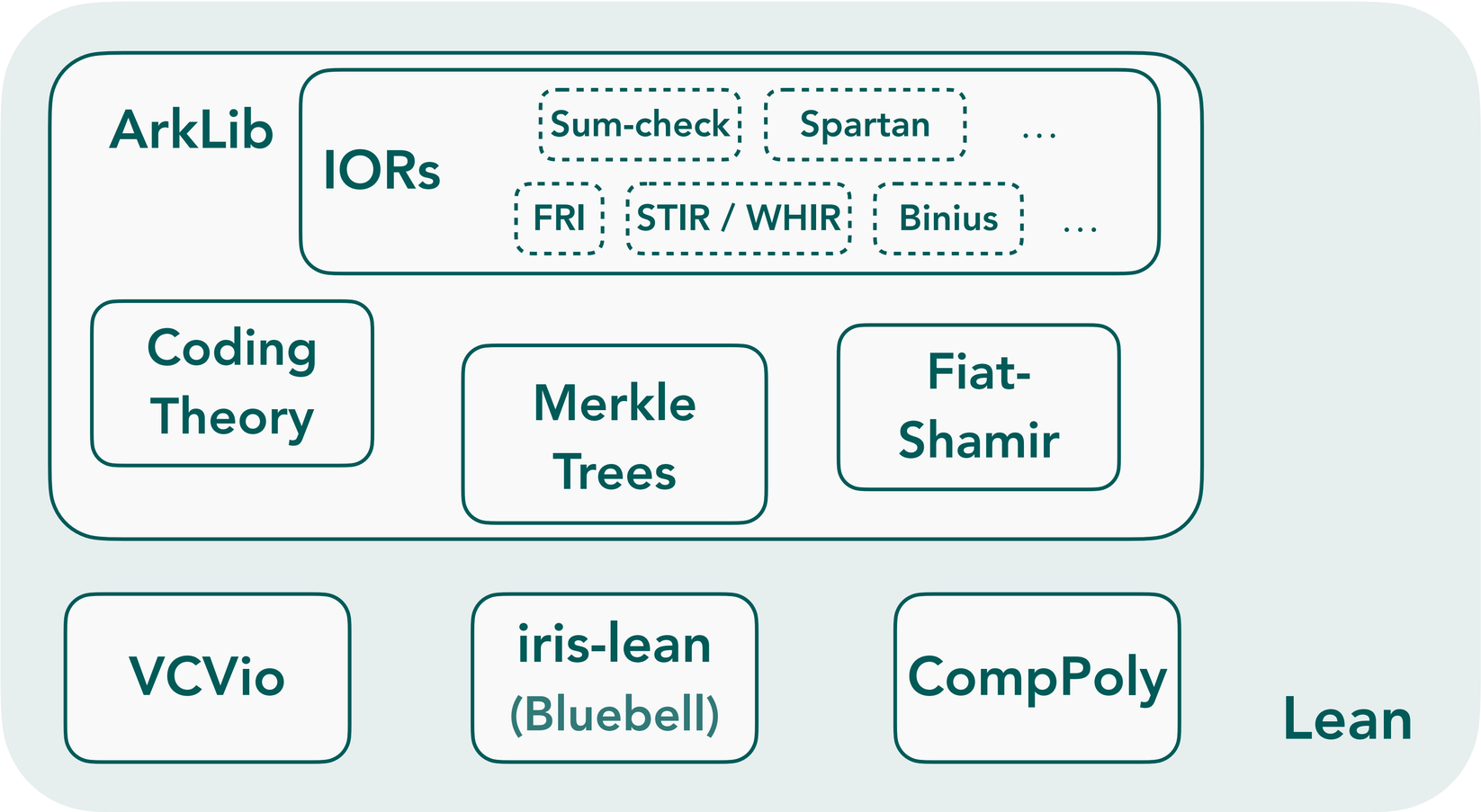
$$\sum_{x' \in \{0,1\}^{n-1}} P(r_1, x') = T_1 \ \wedge \ \text{V hasn't rejected}$$

ArkLib: Current & Future Plans

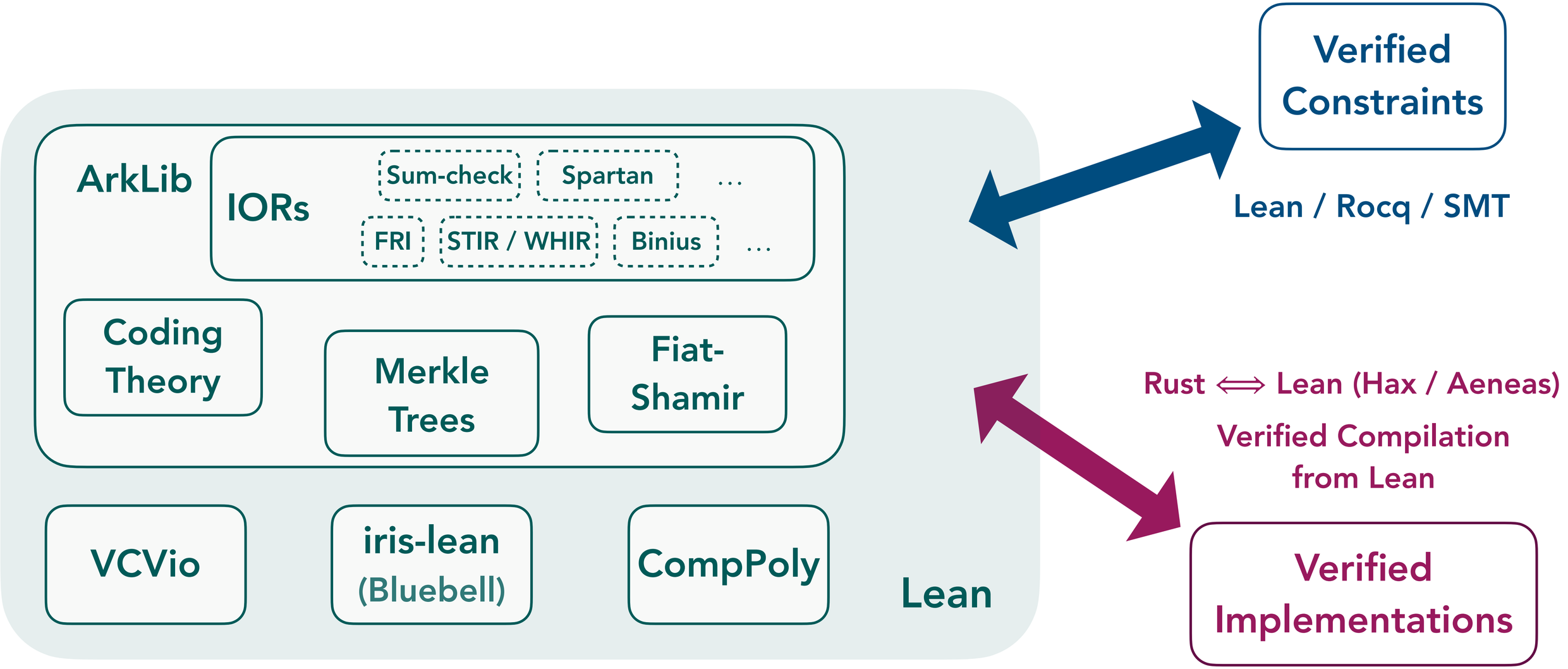
ArkLib: Current & Future Plans



ArkLib: Current & Future Plans



ArkLib: Current & Future Plans



Verified Cryptography with AI

Verified Cryptography with AI

Frontier LLMs are now very good at Lean:

- Claude, GPT, Gemini, etc.
- Harmonic, Logical Intelligence, Math. Inc, AxiomMath, etc.

Verified Cryptography with AI

Frontier LLMs are now very good at Lean:

- Claude, GPT, Gemini, etc.
- Harmonic, Logical Intelligence, Math. Inc, AxiomMath, etc.

We take full advantage of AI in developing ArkLib:

- Generating code / “vibe proving”

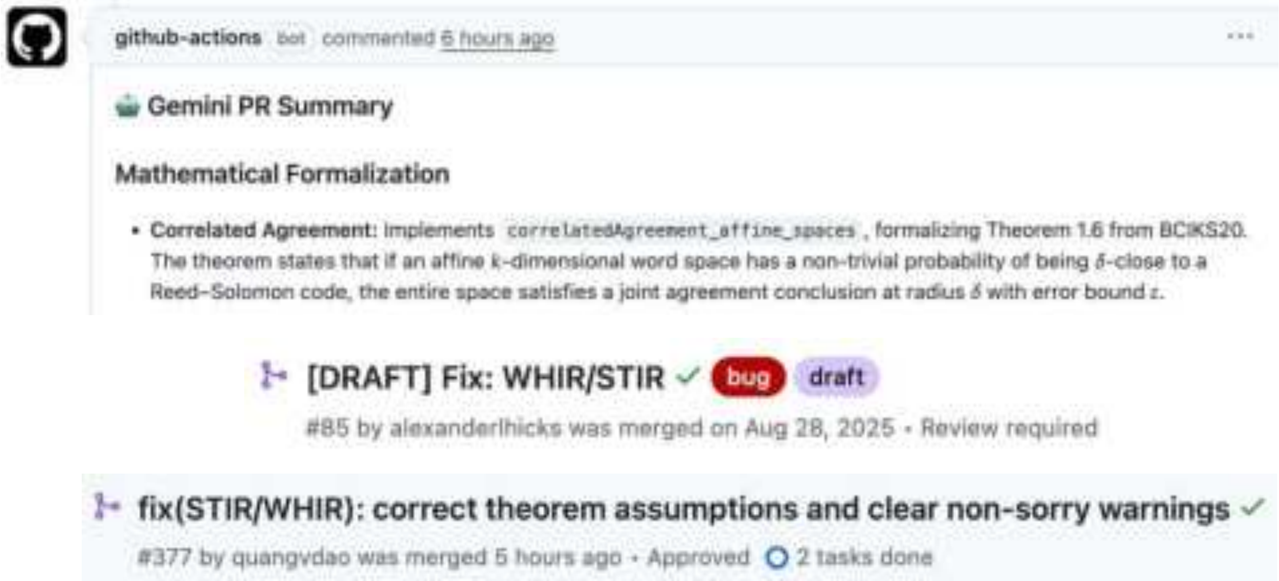
Verified Cryptography with AI

Frontier LLMs are now very good at Lean:

- Claude, GPT, Gemini, etc.
- Harmonic, Logical Intelligence, Math. Inc, AxiomMath, etc.

We take full advantage of AI in developi

- Generating code / “vibe proving”
- PR review - catching misformalization



Verified Cryptography with AI

feat(Existence): prove `E_and_Q_BerlekampWelch_condition`
#358 by pitmonticone was merged 2 weeks ago • Approved

feat(QueryPhase): prove `queryKnowledgeStateFunction`
#357 by pitmonticone was merged 2 weeks ago • Approved

feat(Prelude): prove `fixFirstVariablesOfMQP_degreeLE`
#356 by pitmonticone was merged 2 weeks ago • Approved

.ean:

ıth. Inc

Proof for discr_of_irred_components_nonzero automated-proof
#403 opened 6 hours ago by aleph-prover bot • Review required

Proof for correlatedAgreement_affine_spaces automated-proof
#402 opened 6 hours ago by aleph-prover bot • Review required

Proof for exists_a_set_and_a_matching_polynomial automated-proof
#401 opened 6 hours ago by aleph-prover bot • Review required

We take full advantage of AI in developi

- Generating code / “vibe proving”
- PR review - catching misformalization
- Automated proofs

github-actions bot commented 6 hours ago

Gemini PR Summary

Mathematical Formalization

- **Correlated Agreement:** Implements `correlatedAgreement_affine_spaces`, formalizing Theorem 1.6 from BCIKS20. The theorem states that if an affine k-dimensional word space has a non-trivial probability of being δ-close to a Reed–Solomon code, the entire space satisfies a joint agreement conclusion at radius δ with error bound ε.

[DRAFT] Fix: WHIR/STIR bug draft
#85 by alexanderlhicks was merged on Aug 28, 2025 • Review required

fix(STIR/WHIR): correct theorem assumptions and clear non-sorry warnings
#377 by quangvdao was merged 5 hours ago • Approved 2 tasks done

Summary

Summary

SNARKs are becoming critical cryptographic infrastructure

…and yet are still buggy

Summary

SNARKs are becoming critical cryptographic infrastructure

...and yet are still buggy

ArkLib provides a Lean framework for end-to-end formally verified SNARKs

ArkLib

Summary

SNARKs are becoming critical cryptographic infrastructure

...and yet are still buggy

ArkLib provides a Lean framework for end-to-end formally verified SNARKs

ArkLib

AI now rapidly accelerates progress in ArkLib & verified cryptography in Lean

Summary

SNARKs are becoming critical cryptographic infrastructure

...and yet are still buggy

ArkLib provides a Lean framework for end-to-end formally verified SNARKs

ArkLib

AI now rapidly accelerates progress in ArkLib & verified cryptography in Lean

Thank You!